PROGRAMMING LANGUAGE PRAGMATICS

FOURTH EDITION

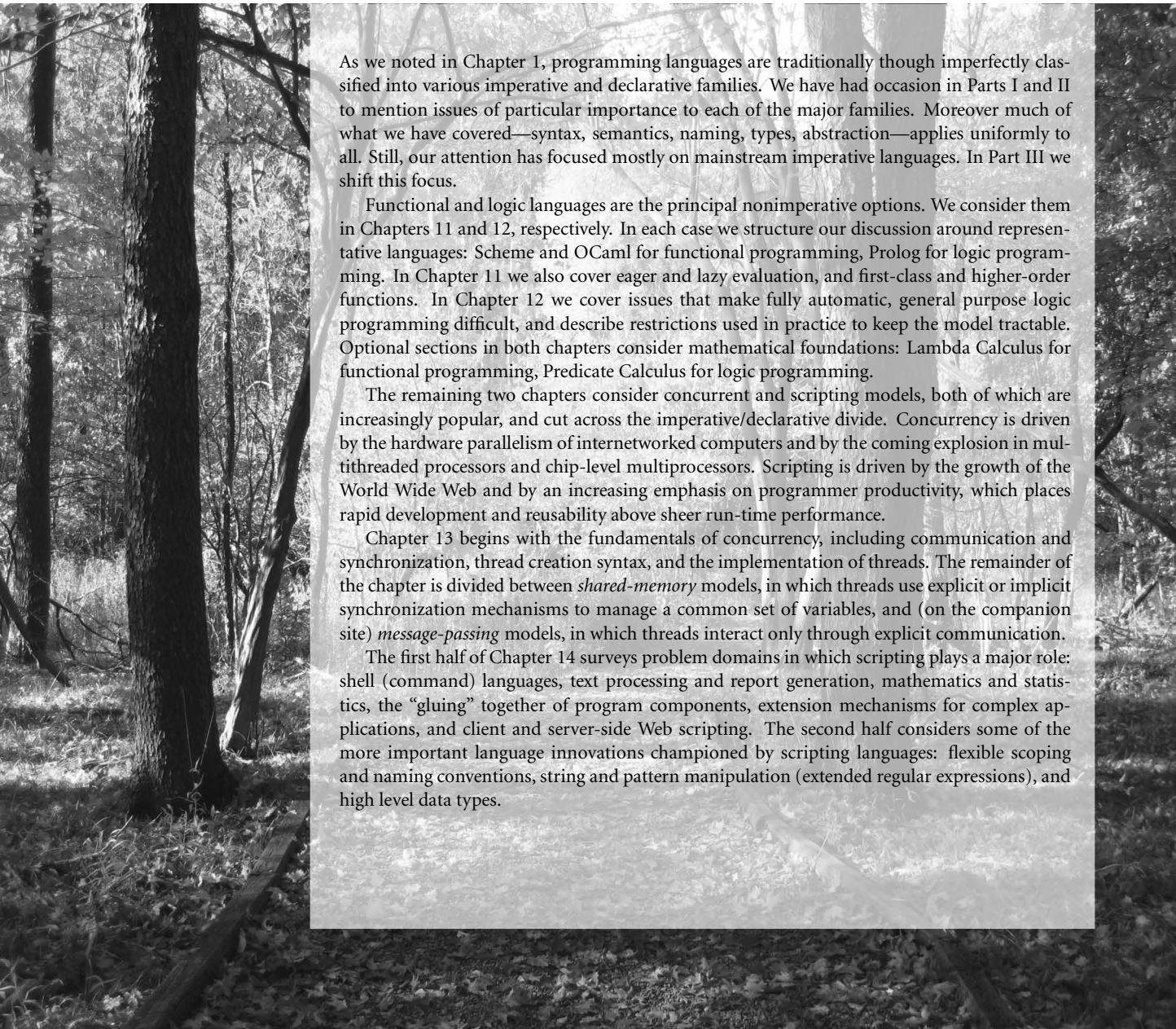
Michael L. Scott

MORGAN KAUFMANN

Programming Language Pragmatics

FOURTH EDITION

III Alternative Programming Models



As we noted in Chapter 1, programming languages are traditionally though imperfectly classified into various imperative and declarative families. We have had occasion in Parts I and II to mention issues of particular importance to each of the major families. Moreover much of what we have covered—syntax, semantics, naming, types, abstraction—applies uniformly to all. Still, our attention has focused mostly on mainstream imperative languages. In Part III we shift this focus.

Functional and logic languages are the principal nonimperative options. We consider them in Chapters 11 and 12, respectively. In each case we structure our discussion around representative languages: Scheme and OCaml for functional programming, Prolog for logic programming. In Chapter 11 we also cover eager and lazy evaluation, and first-class and higher-order functions. In Chapter 12 we cover issues that make fully automatic, general purpose logic programming difficult, and describe restrictions used in practice to keep the model tractable. Optional sections in both chapters consider mathematical foundations: Lambda Calculus for functional programming, Predicate Calculus for logic programming.

The remaining two chapters consider concurrent and scripting models, both of which are increasingly popular, and cut across the imperative/declarative divide. Concurrency is driven by the hardware parallelism of internetworked computers and by the coming explosion in multithreaded processors and chip-level multiprocessors. Scripting is driven by the growth of the World Wide Web and by an increasing emphasis on programmer productivity, which places rapid development and reusability above sheer run-time performance.

Chapter 13 begins with the fundamentals of concurrency, including communication and synchronization, thread creation syntax, and the implementation of threads. The remainder of the chapter is divided between *shared-memory* models, in which threads use explicit or implicit synchronization mechanisms to manage a common set of variables, and (on the companion site) *message-passing* models, in which threads interact only through explicit communication.

The first half of Chapter 14 surveys problem domains in which scripting plays a major role: shell (command) languages, text processing and report generation, mathematics and statistics, the "gluing" together of program components, extension mechanisms for complex applications, and client and server-side Web scripting. The second half considers some of the more important language innovations championed by scripting languages: flexible scoping and naming conventions, string and pattern manipulation (extended regular expressions), and high level data types.

This page intentionally left blank

13 Concurrency

The bulk of this text has focused, implicitly, on *sequential* programs: programs with a single active execution context. As we saw in Chapter 6, sequentiality is fundamental to imperative programming. It also tends to be implicit in declarative programming, partly because practical functional and logic languages usually include some imperative features, and partly because people tend to develop imperative implementations and mental models of declarative programs (applicative order reduction, backward chaining with backtracking), even when language semantics do not require such a model.

By contrast, a program is said to be *concurrent* if it may have more than one active execution context—more than one "thread of control." Concurrency has at least three important motivations:

1. *To capture the logical structure of a problem.* Many programs, particularly servers and graphical applications, must keep track of more than one largely independent "task" at the same time. Often the simplest and most logical way to structure such a program is to represent each task with a separate thread of control. We touched on this "multithreaded" structure when discussing coroutines (Section 9.5) and events (Section 9.6); we will return to it in Section 13.1.1.
2. *To exploit parallel hardware, for speed.* Long a staple of high-end servers and supercomputers, multiple processors (or multiple cores within a processor) have become ubiquitous in desktop, laptop, and mobile devices. To use these cores effectively, programs must generally be written (or rewritten) with concurrency in mind.
3. *To cope with physical distribution.* Applications that run across the Internet or a more local group of machines are inherently concurrent. So are many embedded applications: the control systems of a modern automobile, for example, may span dozens of processors spread throughout the vehicle.

In general, we use the word *concurrent* to characterize any system in which two or more tasks may be underway (at an unpredictable point in their execution) at the same time. Under this definition, coroutines are not concurrent, because at

any given time, all but one of them is stopped at a well-known place. A concurrent system is *parallel* if more than one task can be physically *active* at once; this requires more than one processor. The distinction is purely an implementation and performance issue: from a semantic point of view, there is no difference between true parallelism and the "quasiparallelism" of a system that switches between tasks at unpredictable times. A parallel system is *distributed* if its processors are associated with people or devices that are physically separated from one another in the real world. Under these definitions, "concurrent" applies to all three motivations above. "Parallel" applies to the second and third; "distributed" applies to only the third.

We will focus in this chapter on concurrency and parallelism. Parallelism has become a pressing concern since 2005 or so, with the proliferation of multicore processors. We will have less occasion to touch on distribution. While languages have been designed for distributed computing, most distributed systems run separate programs on every networked processor, and use message-passing library routines to communicate among them.

We begin our study with an overview of the ways in which parallelism may be used in modern programs. Our overview will touch on the motivation for concurrency (even on uniprocessors) and the concept of *races*, which are the principal source of complexity in concurrent programs. We will also briefly survey the architectural features of modern multicore and multiprocessor machines. In Section 13.2 we consider the contrast between shared-memory and message-passing models of concurrency, and between language and library-based implementations. Building on coroutines, we explain how a language or library can create and schedule threads. Section 13.3 focuses on low-level mechanisms for shared-memory synchronization. Section 13.4 extends the discussion to language-level constructs. Message-passing models of concurrency are considered in Section 13.5 (mostly on the companion site).

13.1 Background and Motivation

Concurrency is not a new idea. Much of the theoretical groundwork was laid in the 1960s, and Algol 68 includes concurrent programming features. Widespread interest in concurrency is a relatively recent phenomenon, however; it stems in part from the availability of low-cost multicore and multiprocessor machines, and in part from the proliferation of graphical, multimedia, and web-based applications, all of which are naturally represented by concurrent threads of control.

Levels of Parallelism

Parallelism arises at every level of a modern computer system. It is comparatively easy to exploit at the level of circuits and gates, where signals can propagate down thousands of connections at once. As we move up first to processors and cores, and then to the many layers of software that run on top of them, the *granularity*

of parallelism—the size and complexity of tasks—increases at every level, and it becomes increasingly difficult to figure out what work should be done by each task and how tasks should coordinate.

For 40 years, microarchitectural research was largely devoted to finding more and better ways to exploit the *instruction-level* parallelism (ILP) available in machine language programs. As we saw in Chapter 5, the combination of deep, superscalar pipelines and aggressive speculation allows a modern processor to track dependences among hundreds of "in-flight" instructions, make progress on scores of them, and complete several in every cycle. Shortly after the turn of the century, it became apparent that a limit had been reached: there simply wasn't any more instruction-level parallelism available in conventional programs.

At the next higher level of granularity, so-called *vector parallelism* is available in programs that perform operations repeatedly on every element of a very large data set. Processors designed to exploit this parallelism were the dominant form of supercomputer from the late 1960s through the early 1990s. Their legacy lives on in the vector instructions of mainstream processors (e.g., the MMX, SSE, and AVX extensions to the x86 instruction set), and in modern graphical processing units (GPUs), whose peak performance can exceed that of the typical CPU (central processing unit—a conventional core) by a factor of more than 100.

Unfortunately, vector parallelism arises in only certain kinds of programs. Given the end of ILP, and the limits on clock frequency imposed by heat dissipation (Section C-5.4.4), general-purpose computing today must obtain its performance improvements from *multicore* processors, which require coarser-grain *thread-level* parallelism. The move to multicore has thus entailed a fundamental shift in the nature of programming: where parallelism was once a largely invisible implementation detail, it must now be written explicitly into high-level program structure.

Levels of Abstraction

On today's multicore machines, different kinds of programmers need to understand concurrency at different levels of detail, and use it in different ways.

The simplest, most abstract case arises when using "black box" parallel libraries. A sorting routine or a linear algebra package, for example, may execute in parallel without its caller needing to understand how. In the database world, queries expressed in SQL (Structured Query Language) often execute in parallel as well. Microsoft's .NET Framework includes a *Language-Integrated Query* mechanism (LINQ) that allows database-style queries to be made of program data structures, again with parallelism "under the hood."

At a slightly less abstract level, a programmer may know that certain tasks are mutually independent (because, for example, they access disjoint sets of

EXAMPLE 13.1
Independent tasks in C#

variables). Such tasks can safely execute in parallel.[1] In C#, for example, we can write the following using the Task Parallel Library:

```
Parallel.For(0, 100, i => { A[i] = foo(A[i]); });
```

The first two arguments to `Parallel.For` are "loop" bounds; the third is a delegate, here written as a lambda expression. Assuming `A` is a 100-element array, and that the invocations of `foo` are truly independent, this code will have the same effect as the obvious traditional `for` loop, except that it will run faster, making use of as many cores as possible (up to 100). ■

If our tasks are *not* independent, it may still be possible to run them in parallel if we explicitly *synchronize* their interactions. Synchronization serves to eliminate *races* between threads by controlling the ways in which their actions can interleave in time. Suppose function `foo` in the previous example subtracts 1 from `A[i]` and also counts the number of times that the result is zero. Naively we might implement `foo` as

EXAMPLE 13.2
A simple race condition

```
int zero_count;
public static int foo(int n) {
    int rtn = n - 1;
    if (rtn == 0) zero_count++;
    return rtn;
}
```

Consider now what may happen when two or more instances of this code run concurrently:

Thread 1	**Thread 2**
. . .	. . .
r1 := zero_count	r1 := zero_count
r1 := r1 + 1	r1 := r1 + 1
zero_count := r1	zero_count := r1
. . .	. . .

If the instructions interleave roughly as shown, both threads may load the same value of zero_count, both may increment it by one, and both may store the (only one greater) value back into zero_count. The result may be less than what we expect.

In general, a *race condition* occurs whenever two or more threads are "racing" toward points in the code at which they touch some common object, and the behavior of the system depends on which thread gets there first. In this particular example, the store of zero_count in Thread 1 is racing with the load in Thread 2.

---

1 Ideally, we might like the compiler to figure this out automatically, but the problem of independence is undecidable in the general case.

If Thread 1 gets there first, we will get the "right" result; if Thread 2 gets there first, we won't.

The most common purpose of synchronization is to make some sequence of instructions, known as a *critical section*, appear to be *atomic*—to happen "all at once" from the point of view of every other thread. In our example, the critical section is a load, an increment, and a store. The most common way to make the sequence atomic is with a *mutual exclusion lock*, which we *acquire* before the first instruction of the sequence and *release* after the last. We will study locks in Sections 13.3.1 and 13.3.5. In Sections 13.3.2 and 13.4.4 we will also consider mechanisms that achieve atomicity without locks.

At lower levels of abstraction, expert programmers may need to understand hardware and run-time systems in sufficient detail to *implement* synchronization mechanisms. This chapter should convey a sense of the issues, but a full treatment at this level is beyond the scope of the current text.

13.1.1 The Case for Multithreaded Programs

Our first motivation for concurrency—to capture the logical structure of certain applications—has arisen several times in earlier chapters. In Section C-8.7.1 we noted that interactive I/O must often interrupt the execution of the current program. In a video game, for example, we must handle keystrokes and mouse or joystick motions while continually updating the image on the screen. The standard way to structure such a program, as described in Section 9.6.2, is to execute the input handlers in a separate thread of control, which coexists with one or more threads responsible for updating the screen. In Section 9.5, we considered a screen saver program that used coroutines to interleave "sanity checks" on the file system with updates to a moving picture on the screen. We also considered discrete-event simulation, which uses coroutines to represent the active entities of some real-world system.

The semantics of discrete-event simulation require that events occur atomically at fixed points in time. Coroutines provide a natural implementation, because they execute one at a time: so long as we never switch coroutines in the middle of a to-be-atomic operation, all will be well. In our other examples, however—and indeed in most "naturally concurrent" programs—there is no need for coroutine semantics. By assigning concurrent tasks to threads instead of to coroutines, we acknowledge that those tasks can proceed in parallel if more than one core is available. We also move responsibility for figuring out which thread should run when from the programmer to the language implementation. In return, we give up any notion of trivial atomicity.

EXAMPLE 13.3
Multithreaded web browser

The need for multithreaded programs is easily seen in web-based applications. In a browser such as Chrome or Firefox (see Figure 13.1), there are typically many different threads simultaneously active, each of which is likely to communicate with a remote (and possibly very slow) server several times before completing its task. When the user clicks on a link, the browser creates a thread to request the

specified document. For all but the tiniest pages, this thread will then receive a series of message "packets." As these packets begin to arrive the thread must format them for presentation on the screen. The formatting task is akin to typesetting: the thread must access fonts, assemble words, and break the words into lines. For many special tags within the page, the formatting thread will spawn additional threads: one for each image, one for the background if any, one to format each table, and possibly more to handle separate frames. Each spawned thread will communicate with the server to obtain the information it needs (e.g., the contents of an image) for its particular task. The user, meanwhile, can access items in menus to create new browser windows, edit bookmarks, change preferences, and so on, all in "parallel" with the rendering of page elements. ■

The use of many threads ensures that comparatively fast operations (e.g., display of text) do not wait for slow operations (e.g., display of large images). Whenever one thread *blocks* (waits for a message or I/O), the run-time or operating system will automatically switch execution on the core to run a different thread. In a *preemptive* thread package, these *context switches* will occur at other times as well, to prevent any one thread from hogging processor resources. Any reader who remembers early, more sequential browsers will appreciate the difference that multithreading makes in perceived performance and responsiveness, even on a single-core machine.

The Dispatch Loop Alternative

EXAMPLE 13.4
Dispatch loop web browser

Without language or library support for threads, a browser must either adopt a more sequential structure, or centralize the handling of all delay-inducing events in a single *dispatch loop* (see Figure 13.2). Data structures associated with the dispatch loop keep track of all the tasks the browser has yet to complete. The state of a task may be quite complicated. For the high-level task of rendering a page, the state must indicate which packets have been received and which are still outstanding. It must also identify the various subtasks of the page (images, tables, frames, etc.) so that we can find them all and reclaim their state if the user clicks on a "stop" button.

To guarantee good interactive response, we must make sure that no subaction of `continue_task` takes very long to execute. Clearly we must end the current action whenever we wait for a message. We must also end it whenever we read from a file, since disk operations are slow. Finally, if any task needs to compute for longer than about a tenth of a second (the typical human perceptual threshold), then we must divide the task into pieces, between which we save state and return to the top of the loop. These considerations imply that the condition at the top of the loop must cover the full range of asynchronous events, and that evaluations of the condition must be interleaved with continued execution of any tasks that were subdivided due to lengthy computation. (In practice we would probably need a more sophisticated mechanism than simple interleaving to ensure that neither input-driven nor compute-bound tasks hog more than their share of resources.) ■

```
procedure parse_page(address : url)
    contact server, request page contents
    parse_html_header()
    while current_token in {"<p>", "<h1>", "<ul>", . . . ,
            "<background", "<image", "<table", "<frameset", . . . }
        case current_token of
            "<p>"     :  break_paragraph()
            "<h1>"    :  format_heading(); match("</h1>")
            "<ul>"    :  format_list(); match("</ul>")
            . . .
            "<background" :
                        a : attributes := parse_attributes()
                        fork render_background(a)
            "<image"  :  a : attributes := parse_attributes()
                        fork render_image(a)
            "<table"  :  a : attributes := parse_attributes()
                        scan forward for "</table>" token
                        token_stream s :=. . .    -- table contents
                        fork format_table(s, a)
            "<frameset" :
                        a : attributes := parse_attributes()
                        parse_frame_list(a)
                        match("</frameset>")
        . . .
    . . .
procedure parse_frame_list(a1 : attributes)
    while current_token in {"<frame", "<frameset", "<noframes>"}
        case current_token of
            "<frame"  :  a2 : attributes := parse_attributes()
                        fork format_frame(a1, a2)
        . . .
```

Figure 13.1 Thread-based code from a hypothetical Web browser. To first approximation, the parse_page subroutine is the root of a recursive descent parser for HTML. In several cases, however, the actions associated with recognition of a construct (background, image, table, frameset) proceed concurrently with continued parsing of the page itself. In this example, concurrent threads are created with the fork operation. An additional thread would likely execute in response to keyboard and mouse events.

The principal problem with a dispatch loop—beyond the complexity of subdividing tasks and saving state—is that it hides the algorithmic structure of the program. Every distinct task (retrieving a page, rendering an image, walking through nested menus) could be described elegantly with standard control-flow mechanisms, if not for the fact that we must return to the top of the dispatch loop at every delay-inducing operation. In effect, the dispatch loop turns the program "inside out," making the management of tasks explicit and the control flow within tasks implicit. The resulting complexity is similar to what we encountered when

```
type task_descriptor = record
    -- fields in lieu of thread-local variables, plus control-flow information
    ...
ready_tasks : queue of task_descriptor
...
procedure dispatch()
    loop
        -- try to do something input-driven
        if a new event E (message, keystroke, etc.) is available
            if an existing task T is waiting for E
                continue_task(T, E)
            else if E can be handled quickly, do so
            else
                allocate and initialize new task T
                continue_task(T, E)
        -- now do something compute bound
            if ready_tasks is nonempty
                continue_task(dequeue(ready_tasks), 'ok')

procedure continue_task(T : task, E : event)
    if T is rendering an image
        and E is a message containing the next block of data
            continue_image_render(T, E)
    else if T is formatting a page
        and E is a message containing the next block of data
            continue_page_parse(T, E)
    else if T is formatting a page
        and E is 'ok'       -- we're compute bound
            continue_page_parse(T, E)
    else if T is reading the bookmarks file
        and E is an I/O completion event
            continue_goto_page(T, E)
    else if T is formatting a frame
        and E is a push of the "stop" button
            deallocate T and all tasks dependent upon it
    else if E is the "edit preferences" menu item
        edit_preferences(T, E)
    else if T is already editing preferences
        and E is a newly typed keystroke
            edit_preferences(T, E)
    ...
```

Figure 13.2 Dispatch loop from a hypothetical non-thread-based Web browser. The clauses in continue_task must cover all possible combinations of task state and triggering event. The code in each clause performs the next coherent unit of work for its task, returning when (1) it must wait for an event, (2) it has consumed a significant amount of compute time, or (3) the task is complete. Prior to returning, respectively, code (1) places the task in a dictionary (used by dispatch) that maps awaited events to the tasks that are waiting for them, (2) enqueues the task in ready_tasks, or (3) deallocates the task.

trying to enumerate a recursive set with iterator objects in Section 6.5.3, only worse. Like true iterators, a thread package turns the program "right side out," making the management of tasks (threads) implicit and the control flow within threads explicit.

13.1.2 Multiprocessor Architecture

Parallel computer hardware is enormously diverse. A *distributed* system—one that we think of in terms of interactions among separate programs running on separate machines—may be as large as the Internet, or as small as the components of a cell phone. A parallel but *nondistributed* system—one that we think of in terms of a single program running on a single machine—may still be very large. China's Tianhe-2 supercomputer, for example, has more than 3 million cores, consumes over 17 MW of power, and occupies 720 square meters of floor space (about a fifth of an acre).

Historically, most parallel but nondistributed machines were *homogeneous*—their processors were all identical. In recent years, many machines have added programmable GPUs, first as separate processors, and more recently as separate portions of a single processor chip. While the cores of a GPU are internally homogeneous, they are very different from those of the typical CPU, leading to a globally *heterogeneous* system. Future systems may have cores of many other kinds as well, each specialized to particular kinds of programs or program components.

In an ideal world, programming languages and runtimes would map program fragments to suitable cores at suitable times, but this sort of automation is still very much a research goal. As of 2015, programmers who want to make use of the GPU write appropriate portions of their code in special-purpose languages like OpenCL or CUDA, which emphasize repetitive operations over vectors. A main program, running on the CPU, then ships the resulting "kernels" to the GPU explicitly.

In the remainder of this chapter, we will concentrate on thread-level parallelism for homogeneous machines. For these, many of the most important architectural questions involve the memory system. In some machines, all of physical memory is accessible to every core, and the hardware guarantees that every write is quickly visible everywhere. At the other extreme, some machines partition main memory among processors, forcing cores to interact through some separate message-passing mechanism. In intermediate designs, some machines share memory in a *noncoherent* fashion, making writes on one core visible to another only when both have explicitly flushed their caches.

From the point of view of language or library implementation, the principal distinction between shared-memory and message-passing hardware is that messages typically require the active participation of cores at both ends of the connection: one to send, the other to receive. On a shared-memory machine, a core can read and write remote memory without any other core's assistance.

On small machines (2–4 processors, say), main memory may be *uniform*—equally distant from all processors. On larger machines (and even on some very

small machines), memory may be *nonuniform* instead—each bank may be physically adjacent to a particular processor or small group of processors. Cores in any processor can then access the memory of any other, but local memory is faster. Assuming all memory is cached, of course, the difference appears only on cache misses, where the penalty for local memory is lower.

Memory Coherence

As suggested by the notion of noncoherent memory, caches introduce a serious problem for shared-memory machines: unless we do something special, a core that has cached a particular memory location may run for an arbitrarily long time without seeing changes that have been made to that location by other cores. This problem—how to keep cached copies of a memory location consistent with

EXAMPLE 13.5
The cache coherence problem

DESIGN & IMPLEMENTATION

13.1 What, exactly, is a processor?

From roughly 1975 to 2005, a processor typically ran only one thread at a time, and occupied one full chip. Today, most vendors still use the term "processor" to refer to the physical device that "does the computing," and whose pins connect it to the rest of the computer, but the internal structure is much more complicated: there may be more than one chip inside the physical package, each chip may have multiple *cores* (each of which would have been called a "processor" in previous hardware generations), and each core may have multiple *hardware threads* (independent register sets, which allow the core's pipeline(s) to run a mix of instructions drawn from multiple software threads). A modern processor may also include many megabytes of on-chip cache, organized into multiple levels, and physically distributed and shared among the cores in complicated ways. Increasingly, processors may incorporate on-chip memory controllers, network interfaces, graphical processing units, or other formerly "peripheral" components, making continued use of the term "processor" problematic but no less common.

From a software perspective, the good news is that operating systems and programming languages generally model every concurrent activity as a thread, regardless of whether it shares a core, a chip, or a package with other threads. We will follow this convention for most of the rest of this chapter, ignoring the complexity of the underlying hardware. When we need to refer to the hardware on which a thread runs, we will usually call it a "core." The bad news is that a model of computing in which "everything is just a thread" hides details that are crucial to understanding and improving performance. Future chips are likely to include ever larger numbers of heterogeneous cores and complex on-chip networks. To use these chips effectively, language implementations will need to become much more sophisticated about scheduling threads onto the underlying hardware. How much of the task will need to be visible to the application programmer remains to be determined.

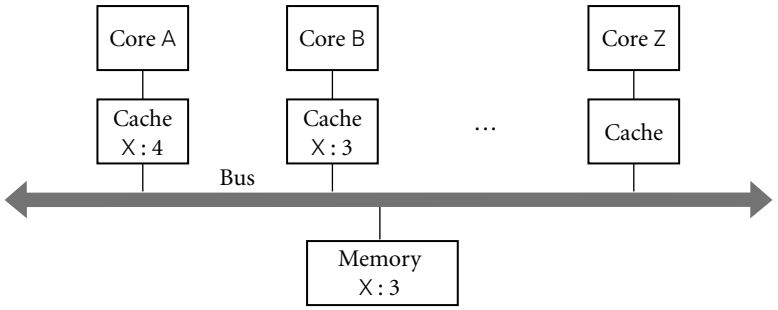


Figure 13.3 The cache coherence problem for shared-memory multicore and multiprocessor machines. Here cores A and B have both read variable X from memory. As a side effect, a copy of X has been created in the cache of each core. If A now changes X to 4 and B reads X again, how do we ensure that the result is a 4 and not the still-cached 3? Similarly, if Z reads X into its cache, how do we ensure that it obtains the 4 from A's cache instead of the stale 3 from memory?

one another—is known as the *coherence* problem (see Figure 13.3). On a simple bus-based machine, the problem is relatively easy to solve: the broadcast nature of the communication medium allows cache controllers to eavesdrop (*snoop*) on the memory traffic of other cores. When a core needs to write a cache line, it requests an *exclusive* copy, and waits for other cores to *invalidate* their copies. On a bus the waiting is trivial, and the natural ordering of messages determines who wins in the event of near-simultaneous requests. Cores that try to access a line in the wake of invalidation must go back to memory (or to another core's cache) to obtain an up-to-date copy.

Bus-based cache coherence algorithms are now a standard, built-in part of most commercial microprocessors. On large machines, the lack of a broadcast bus makes cache coherence a significantly more difficult problem; commercial implementations are available, but they are complex and expensive. On both small and large machines, the fact that coherence is not instantaneous (it takes time for notifications to propagate) means that we must consider the *order* in which updates to different locations appear to occur from the point of view of different processors. Ensuring a *consistent* view is a surprisingly difficult problem; we will return to it in Section 13.3.3.

As of 2015, there are multicore versions of every major instruction set architecture, including ARM, x86, Power, SPARC, x86-64, and IA-64 (Itanium). Small, cache-coherent multiprocessors built from these are available from dozens of manufacturers. Larger, cache-coherent shared-memory multiprocessors are available from several manufacturers, including Oracle, HP, IBM, and SGI.

Supercomputers

Though dwarfed financially by the rest of the computer industry, supercomputing has always played a disproportionate role in the development of computer

technology and the advancement of human knowledge. Supercomputers have changed dramatically over time, and they continue to evolve at a very rapid pace. They have always, however, been parallel machines.

Because of the complexity of cache coherence, it is difficult to build large shared-memory machines. SGI sells machines with as many as 256 processors (2048 cores). Cray builds even larger shared-memory machines, but without the ability to cache remote locations. For the most part, however, the vector supercomputers of the 1960s–80s were displaced not by large multiprocessors, but by modest numbers of smaller multiprocessors or by very large numbers of commodity (mainstream) processors, connected by custom high-performance networks. As network technology "trickled down" into the broader market, these machines in turn gave way to *clusters* composed of both commodity multicore processors and commodity networks (Gigabit Ethernet or Infiniband). As of 2015, clusters have come to dominate everything from modest server farms up to all but the very fastest supercomputer sites. Large-scale on-line services like Google, Amazon, or Facebook are typically backed by clusters with tens or hundreds of thousands of cores (in Google's case, probably millions).

Today's fastest machines are constructed from special high-density multicore chips with low per-core operating power. The Tianhe-2 (the fastest machine in the world as of June 2015) uses a 2:3 mix of Intel 12-core Ivy Bridge and 61-core Phi processors, at 10 W and 5 W per core, respectively. Given current trends, it seems likely that future machines, both high-end and commodity, will be increasingly dense and increasingly heterogeneous.

From a programming language perspective, the special challenge of supercomputing is to accommodate nonuniform access times and (in most cases) the lack of hardware support for shared memory across the full machine. Today's supercomputers are programmed mostly with message-passing libraries (MPI in particular) and with languages and libraries in which there is a clear syntactic distinction between local and remote memory access.

✓ CHECK YOUR UNDERSTANDING

1. Explain the distinctions among *concurrent*, *parallel*, and *distributed*.
2. Explain the motivation for concurrency. Why do people write concurrent programs? What accounts for the increased interest in concurrency in recent years?
3. Describe the implementation levels at which parallelism appears in modern systems, and the levels of abstraction at which it may be considered by the programmer.
4. What is a *race condition*? What is *synchronization*?
5. What is a *context switch*? *Preemption*?
6. Explain the concept of a *dispatch loop*. What are its advantages and disadvantages with respect to multithreaded code?

7. Explain the distinction between a *multiprocessor* and a *cluster*; between a *processor* and a *core*.
8. What does it mean for memory in a multiprocessor to be *uniform*? What is the alternative?
9. Explain the *coherence problem* for multicore and multiprocessor caches.
10. What is a *vector machine*? Where does vector technology appear in modern systems?

13.2 Concurrent Programming Fundamentals

Within a concurrent program, we will use the term *thread* to refer to the active entity that the programmer thinks of as running concurrently with other threads. In most systems, the threads of a given program are implemented on top of one or more *processes* provided by the operating system. OS designers often distinguish between a *heavyweight* process, which has its own address space, and a collection of *lightweight* processes, which may share an address space. Lightweight processes were added to most variants of Unix in the late 1980s and early 1990s, to accommodate the proliferation of shared-memory multiprocessors.

We will sometimes use the word *task* to refer to a well-defined unit of work that must be performed by some thread. In one common programming idiom, a collection of threads shares a common "bag of tasks"—a list of work to be done. Each thread repeatedly removes a task from the bag, performs it, and goes back for another. Sometimes the work of a task entails adding new tasks to the bag.

Unfortunately, terminology is inconsistent across systems and authors. Several languages call their threads processes. Ada calls them tasks. Several operating systems call lightweight processes threads. The Mach OS, from which OSF Unix and Mac OS X are derived, calls the address space shared by lightweight processes a task. A few systems try to avoid ambiguity by coining new words, such as "actors," "fibers," or "filaments." We will attempt to use the definitions of the preceding two paragraphs consistently, and to identify cases in which the terminology of particular languages or systems differs from this usage.

13.2.1 Communication and Synchronization

In any concurrent programming model, two of the most crucial issues to be addressed are *communication* and *synchronization*. Communication refers to any mechanism that allows one thread to obtain information produced by another. Communication mechanisms for imperative programs are generally based on either *shared memory* or *message passing*. In a shared-memory programming model, some or all of a program's variables are accessible to multiple threads.

For a pair of threads to communicate, one of them writes a value to a variable and the other simply reads it. In a pure message-passing programming model, threads have no common state: for a pair of threads to communicate, one of them must perform an explicit send operation to transmit data to another. (Some languages—Ada, Go, and Rust, for example—provide both messages *and* shared memory.)

Synchronization refers to any mechanism that allows the programmer to control the relative order in which operations occur in different threads. Synchronization is generally implicit in message-passing models: a message must be sent before it can be received. If a thread attempts to receive a message that has not yet been sent, it will wait for the sender to catch up. Synchronization is generally not implicit in shared-memory models: unless we do something special, a "receiving" thread could read the "old" value of a variable, before it has been written by the "sender."

In both shared-memory and message-based programs, synchronization can be implemented either by *spinning* (also called *busy-waiting*) or by *blocking*. In busy-wait synchronization, a thread runs a loop in which it keeps reevaluating some condition until that condition becomes true (e.g., until a message queue becomes nonempty or a shared variable attains a particular value)—presumably as a result of action in some other thread, running on some other core. Note that busy-waiting makes no sense on a uniprocessor: we cannot expect a condition to become true while we are monopolizing a resource (the one and only core) required to make it true. (A thread on a uniprocessor may sometimes busy-wait for the completion of I/O, but that's a different situation: the I/O device runs in parallel with the processor.)

In blocking synchronization (also called *scheduler-based* synchronization), the waiting thread voluntarily relinquishes its core to some other thread. Before doing so, it leaves a note in some data structure associated with the synchronization condition. A thread that makes the condition true at some point in the future will find the note and take action to make the blocked thread run again. We will con-

DESIGN & IMPLEMENTATION

13.2 Hardware and software communication

As described in Section 13.1.2, the distinction between shared memory and message passing applies not only to languages and libraries but also to computer hardware. It is important to note that the model of communication and synchronization provided by the language or library need not necessarily agree with that of the underlying hardware. It is easy to implement message passing on top of shared-memory hardware. With a little more effort, one can also implement shared memory on top of message-passing hardware. Systems in this latter camp are sometimes referred to as *software distributed shared memory* (S-DSM).

	Shared memory	Message passing	Distributed computing
Language	Java, C# C/C++11	Go	Erlang
Extension	OpenMP		Remote procedure call
Library	`pthreads`, Windows threads	MPI	Internet libraries

Figure 13.4 **Examples of parallel programming systems.** There is also a very large number of experimental, pedagogical, or niche proposals for each of the regions in the table.

sider synchronization again briefly in Section 13.2.4, and then more thoroughly in Section 13.3.

13.2.2 Languages and Libraries

Thread-level concurrency can be provided to the programmer in the form of explicitly concurrent languages, compiler-supported extensions to traditional sequential languages, or library packages outside the language proper. All three options are widely used, though shared-memory languages are more common at the "low end" (for multicore and small multiprocessor machines), and message-passing libraries are more common at the "high end" (for massively parallel supercomputers). Examples of systems in widespread use are categorized in Figure 13.4.

For many years, almost all parallel programming employed traditional sequential languages (largely C and Fortran) augmented with libraries for synchronization or message passing, and this approach still dominates today. In the Unix world, shared memory parallelism has largely converged on the POSIX `pthreads` standard, which includes mechanisms to create, destroy, schedule, and synchronize threads. This standard became an official part of both C and C++ as of their 2011 versions. Similar functionality for Windows machines is provided by Microsoft's thread package and compilers. For high-end scientific computing, message-based parallelism has likewise converged on the MPI (Message Passing Interface) standard, with open-source and commercial implementations available for almost every platform.

While language support for concurrency goes back all the way to Algol 68 (and coroutines to Simula), and while such support was widely available in Ada by the late 1980s, widespread interest in these features didn't really arise until the mid-1990s, when the explosive growth of the World Wide Web began to drive the development of parallel servers and concurrent client programs. This development coincided nicely with the introduction of Java, and Microsoft followed with C# a few years later. Though not yet as influential, many other languages, including Erlang, Go, Haskell, Rust, and Scala, are explicitly parallel as well.

In the realm of scientific programming, there is a long history of extensions to Fortran designed to facilitate the parallel execution of loop iterations. By the turn of the century this work had largely converged on a set of extensions known as OpenMP, available not only in Fortran but also in C and C++. Syntactically, OpenMP comprises a set of *pragmas* (compiler directives) to create and synchronize threads, and to schedule work among them. On machines composed of a network of multiprocessors, it is increasingly common to see hybrid programs that use OpenMP within a multiprocessor and MPI across them.

In both the shared memory and message passing columns of Figure 13.4, the parallel constructs are intended for use within a single multithreaded program. For communication across program boundaries in distributed systems, programmers have traditionally employed library implementations of the standard Internet protocols, in a manner reminiscent of file-based I/O (Section C-8.7). For client-server interaction, however, it can be attractive to provide a higher-level interface based on *remote procedure calls* (RPC), an alternative we consider further in Section C-13.5.4.

In comparison to library packages, an explicitly concurrent programming language has the advantage of compiler support. It can make use of syntax other than subroutine calls, and can integrate communication and thread management more tightly with such concepts as type checking, scoping, and exceptions. At the same time, since most programs have historically been sequential, concurrent languages have been slow to gain widespread acceptance, particularly given that the presence of concurrent features can sometime make the sequential case more difficult to understand.

13.2.3 Thread Creation Syntax

Almost every concurrent system allows threads to be created (and destroyed) dynamically. Syntactic and semantic details vary considerably from one language or library to another, but most conform to one of six principal options: co-begin, parallel loops, launch-at-elaboration, fork (with optional join), implicit receipt, and early reply. The first two options delimit threads with special control-flow constructs. The others use syntax resembling (or identical to) subroutines.

At least one pedagogical language (SR) provided all six options. Most others pick and choose. Most libraries use fork/join, as do Java and C#. Ada uses both launch-at-elaboration and fork. OpenMP uses co-begin and parallel loops. RPC systems are typically based on implicit receipt.

Co-begin

EXAMPLE 13.6
General form of co-begin

The usual semantics of a compound statement (sometimes delimited with begin...end) call for sequential execution of the constituent statements. A co-begin construct calls instead for concurrent execution:

```
co-begin                    -- all n statements run concurrently
```

```
    stmt_1
    stmt_2
    ...
    stmt_n
end
```

Each statement can itself be a sequential or parallel compound, or (commonly) a subroutine call.

EXAMPLE 13.7
Co-begin in OpenMP

Co-begin was the principal means of creating threads in Algol-68. It appears in a variety of other systems as well, including OpenMP:

```
#pragma omp sections
{
#   pragma omp section
    { printf("thread 1 here\n"); }

#   pragma omp section
    { printf("thread 2 here\n"); }
}
```

In C, OpenMP directives all begin with `#pragma omp`. (The `#` sign must appear in column one.) Most directives, like those shown here, must appear immediately before a loop construct or a compound statement delimited with curly braces.

Parallel Loops

Many concurrent systems, including OpenMP, several dialects of Fortran, and the Task Parallel Library for .NET, provide a loop whose iterations are to be executed concurrently. In OpenMP for C, we might say

EXAMPLE 13.8
A parallel loop in OpenMP

```
#pragma omp parallel for
for (int i = 0; i < 3; i++) {
    printf("thread %d here\n", i);
}
```

EXAMPLE 13.9
A parallel loop in C#

In C# with the Task Parallel Library, the equivalent code looks like this:

```
Parallel.For(0, 3, i => {
    Console.WriteLine("Thread " + i + "here");
});
```

The third argument to `Parallel.For` is a delegate, in this case a lambda expression. A similar `Foreach` method expects two arguments—an iterator and a delegate.

In many systems it is the programmer's responsibility to make sure that concurrent execution of the loop iterations is safe, in the sense that correctness will never depend on the outcome of race conditions. Access to global variables, for example, must generally be synchronized, to make sure that iterations do not

conflict with one another. In a few languages (e.g., Occam), language rules prohibit conflicting accesses. The compiler checks to make sure that a variable written by one thread is neither read nor written by any concurrently active thread. In a similar but slightly more flexible vein, the `do concurrent` loop of Fortran 2008 constitutes an assertion on the programmer's part that iterations of the loop are mutually independent, and hence can safely be executed in any order, or in parallel. Several rules on the content of the loop—some but not all of them enforceable by the compiler—reduce the likelihood that programmers will make this assertion incorrectly.

Historically, several parallel dialects of Fortran provided other forms of parallel loop, with varying semantics. The `forall` loop of High Performance Fortran (HPF) was subsequently incorporated into Fortran 95. Like `do concurrent`, it indicates that iterations can proceed in parallel. To resolve race conditions, however, it imposes automatic, internal synchronization on the constituent statements of the loop, each of which must be an assignment or a nested `forall` loop. Specifically, all reads of variables in a given assignment statement, in all iterations, must occur before any write to the left-hand side, in any iteration. The writes of the left-hand side in turn must occur before any reads in the next assignment statement. In the following example, the first assignment in the loop will read $n-1$ elements of `B` and $n-1$ elements of `C`, and then update $n-1$ elements of `A`. Subsequently, the second assignment statement will read all n elements of `A` and then update $n-1$ of them:

EXAMPLE 13.10
`Forall` in Fortran 95

```
forall (i=1:n-1)
    A(i) = B(i) + C(i)
    A(i+1) = A(i) + A(i+1)
end forall
```

Note in particular that all of the updates to `A(i)` in the first assignment statement occur before any of the reads in the second assignment statement. Moreover in the second assignment statement the update to `A(i+1)` is *not* seen by the read of `A(i)` in the "subsequent" iteration: the iterations occur in parallel and each reads the variables on its right-hand side before updating its left-hand side. ■

For loops that iterate over the elements of an array, the `forall` semantics are ideally suited for execution on a vector machine. For more conventional multiprocessors, HPF provides an extensive set of *data distribution* and *alignment* directives that allow the programmer to scatter elements across the memory associated with a large number of processors. Within a `forall` loop, the computation in a given assignment statement is usually performed by the processor that "owns" the element on the assignment's left-hand side. In many cases an HPF or Fortran 95 compiler can prove that there are no dependences among certain (portions of) constituent statements of a `forall` loop, and can allow them to proceed without actually implementing synchronization.

OpenMP does not enforce the statement-by-statement synchronization of `forall`, but it does provide significant support for scheduling and data management. Optional "clauses" on `parallel` directives can specify how many threads

to create, and which iterations of the loop to perform in which thread. They can also specify which program variables should be shared by all threads, and which should be split into a separate copy for each thread. It is even possible to specify that a private variable should be *reduced* across all threads at the end of the loop, using a commutative operator. To sum the elements of a very large vector, for example, one might write

EXAMPLE 13.11
Reduction in OpenMP

```
double A[N];
...
double sum = 0;
#pragma omp parallel for schedule(static) \
    default(shared) reduction(+:sum)
for (int i = 0; i < N; i++) {
    sum += A[i];
}
printf("parallel sum: %f\n", sum);
```

Here the `schedule(static)` clause indicates that the compiler should divide the iterations evenly among threads, in contiguous groups. So if there are t threads, the first thread should get the first N/t iterations, the second should get the next N/t iterations, and so on. The `default(shared)` clause indicates that all variables (other than `i`) should be shared by all threads, unless otherwise specified. The `reduction(+:sum)` clause makes `sum` an exception: every thread should have its own copy (initialized from the value in effect before the loop), and the copies should be combined (with `+`) at the end. If t is large, the compiler will probably sum the values using a tree of depth $\log(t)$. ■

Launch-at-Elaboration

In several languages, Ada among them, the code for a thread may be declared with syntax resembling that of a subroutine with no parameters. When the declaration is elaborated, a thread is created to execute the code. In Ada (which calls its threads `tasks`) we may write

EXAMPLE 13.12
Elaborated tasks in Ada

```
procedure P is
task T is
...
end T;
begin -- P
...
end P;
```

Task `T` has its own `begin`... `end` block, which it begins to execute as soon as control enters procedure `P`. If `P` is recursive, there may be many instances of `T` at the same time, all of which execute concurrently with each other and with whatever task is executing (the current instance of) `P`. The main program behaves like an initial default task.

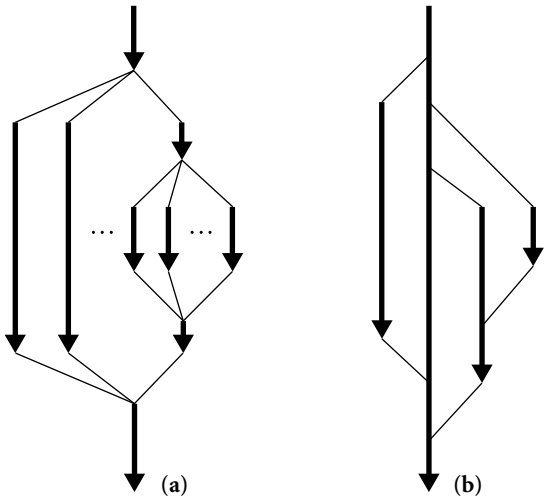


Figure 13.5 Lifetime of concurrent threads. With co-begin, parallel loops, or launch-at-elaboration (a), threads are always properly nested. With fork/join (b), more general patterns are possible.

When control reaches the end of procedure `P`, it will wait for the appropriate instance of `T` (the one that was created at the beginning of this instance of `P`) to complete before returning. This rule ensures that the local variables of `P` (which are visible to `T` under the usual static scope rules) are never deallocated before `T` is done with them. ■

Fork/Join

EXAMPLE 13.13
Co-begin vs fork/join

Co-begin, parallel loops, and launch-at-elaboration all lead to a concurrent control-flow pattern in which thread executions are properly nested (see Figure 13.5a). The fork operation is more general: it makes the creation of threads an explicit, executable operation. The companion join operation, when provided, allows a thread to wait for the completion of a previously forked thread. Because fork and join are not tied to nested constructs, they can lead to arbitrary patterns of concurrent control flow (Figure 13.5b). ■

EXAMPLE 13.14
Task types in Ada

In addition to providing launch-at-elaboration tasks, Ada allows the programmer to define task *types*:

```
task type T is
...
begin
...
end T;
```

The programmer may then declare variables of type `access T` (pointer to `T`), and may create new tasks via dynamic allocation:

```
pt : access T := new T;
```

The `new` operation is a fork: it creates a new thread and starts it executing. There is no explicit join operation in Ada, though parent and child tasks can always synchronize with one another explicitly if desired (e.g., immediately before the child completes its execution). As with launch-at-elaboration, control will wait automatically at the end of any scope in which task types are declared for all threads using the scope to terminate.

Any information an Ada task needs in order to do its job must be communicated through shared variables or through explicit messages sent after the task has started execution. Most systems, by contrast, allow parameters to be passed to a thread at start-up time. In Java one obtains a thread by constructing an object of some class derived from a predefined class called `Thread`:

EXAMPLE 13.15
Thread creation in Java 2

```
class ImageRenderer extends Thread {
    ...
    ImageRenderer( args ) {
        // constructor
    }
```

DESIGN & IMPLEMENTATION

13.3 Task-parallel and data-parallel computing

One of the most basic decisions a programmer has to make when writing a parallel program is how to divide work among threads. One common strategy, which works well on small machines, is to use a separate thread for each of the program's major tasks or functions, and to pipeline or otherwise overlap their execution. In a word processor, for example, one thread might be devoted to breaking paragraphs into lines, another to pagination and figure placement, another to spelling and grammar checking, and another to rendering the image on the screen. This strategy is often known as *task parallelism*. Its principal disadvantage is that it doesn't naturally scale to very large numbers of processors. For that, one generally needs *data parallelism*, in which more or less the same operations are applied concurrently to the elements of some very large data set. An image manipulation program, for example, may divide the screen into *n* small tiles, and use a separate thread to process each tile. A game may use a separate thread for every moving character or object.

A programming system whose features are designed for data parallelism is sometimes referred to as a data-parallel language or library. Task parallel programs are commonly based on co-begin, launch-at-elaboration, or fork/join: the code in different threads can be different. Data parallel programs are commonly based on parallel loops: each thread executes the same code, using different data. OpenCL and CUDA, unsurprisingly, are in the data-parallel camp: programmable GPUs are optimized for data parallel programs.

```
    public void run() {
        // code to be run by the thread
    }
}
...
ImageRenderer rend = new ImageRenderer( constructor_args );
```

Superficially, the use of `new` resembles the creation of dynamic tasks in Ada. In Java, however, the new thread does *not* begin execution when first created. To start it, the parent (or some other thread) must call the method named `start`, which is defined in `Thread`:

```
rend.start();
```

`Start` makes the thread runnable, arranges for it to execute its `run` method, and returns to the caller. The programmer must define an appropriate `run` method in every class derived from `Thread`. The `run` method is meant to be called only by `start`; programmers should not call it directly, nor should they redefine `start`. There is also a `join` method:

```
rend.join();        // wait for completion
```

EXAMPLE 13.16
Thread creation in C#

The constructor for a Java thread typically saves its arguments in fields that are later accessed by `run`. In effect, the class derived from `Thread` functions as an *object closure*, as described in Section 3.6.3. Several languages, Modula-3 and C# among them, use closures more explicitly. Rather than require every thread to be derived from a common `Thread` class, C# allows one to be created from an arbitrary `ThreadStart` delegate:

```
class ImageRenderer {
    ...
    public ImageRenderer( args ) {
        // constructor
    }
    public void Foo() {     // Foo is compatible with ThreadStart;
                            // its name is not significant
        // code to be run by the thread
    }
}
...
ImageRenderer rendObj = new ImageRenderer( constructor_args );
Thread rend = new Thread(new ThreadStart(rendObj.Foo));
```

If thread arguments can be gleaned from the local context, this can even be written as

```
Thread rend = new Thread(delegate() {
    // code to be run by the thread
});
```

(Remember, C# has unlimited extent for anonymous delegates.) Either way, the new thread is started and awaited just as it is in Java:

```
rend.Start();
...
rend.Join();
```

EXAMPLE 13.17
Thread pools in Java 5

As of Java 5 (with its `java.util.concurrent` library), programmers are discouraged from creating threads explicitly. Rather, tasks to be accomplished are represented by objects that support the `Runnable` interface, and these are passed to an `Executor` object. The `Executor` in turn farms them out to a managed pool of threads:

```
class ImageRenderer implements Runnable {
    ...
    // constructor and run() method same as before
}
...
Executor pool = Executors.newFixedThreadPool(4);
...
pool.execute(new ImageRenderer( constructor_args ));
```

Here the argument to `newFixedThreadPool` (one of a large number of standard `Executor` *factories*) indicates that `pool` should manage four threads. Each task specified in a call to `pool.execute` will be run by one of these threads. By separating the concepts of task and thread, Java allows the programmer (or run-time code) to choose an `Executor` class whose level of true concurrency and scheduling discipline are appropriate to the underlying OS and hardware. (In this example we have used a particularly simple pool, with exactly four threads.) C# has similar thread pool facilities. Like C# threads, they are based on delegates.

EXAMPLE 13.18
`Spawn` and `sync` in Cilk

A particularly elegant realization of `fork` and `join` appears in the Cilk programming language, developed by researchers at MIT, and subsequently developed into a commercial venture acquired by Intel. To fork a logically concurrent task in Cilk, one simply prepends the keyword `spawn` to an ordinary function call:

```
spawn foo( args );
```

At some later time, invocation of the built-in operation `sync` will join with all tasks previously spawned by the calling task. The principal innovation of Cilk is the mechanism for scheduling tasks. The language implementation includes

a highly efficient thread pool mechanism that explores the task-creation graph depth first with a near-minimal number of context switches and automatic load balancing across threads. Java 7 added a similar but more restricted mechanism in the form of a `ForkJoinPool` for the `Executor` service. ■

Implicit Receipt

We have assumed in all our examples so far that newly created threads will run in the address space of the creator. In RPC systems it is often desirable to create a new thread automatically in response to an incoming request from some *other* address space. Rather than have an existing thread execute a receive operation, a server can *bind* a communication channel to a local thread body or subroutine. When a request comes in, a new thread springs into existence to handle it.

In effect, the `bind` operation grants remote clients the ability to perform a `fork` within the server's address space, though the process is often less than fully automatic. We will consider RPC in more detail in Section C-13.5.4.

Early Reply

EXAMPLE 13.19
Modeling subroutines with fork/join

We normally think of sequential subroutines in terms of a single thread, which saves its current context (its program counter and registers), executes the subroutine, and returns to what it was doing before. The effect is the same, however, if we have two threads—one that executes the caller and another that executes the callee. In this case, the call is essentially a fork/join pair. The caller waits for the callee to terminate before continuing execution. ■

Nothing dictates, however, that the callee has to terminate in order to release the caller; all it really has to do is complete the portion of its work on which re-

DESIGN & IMPLEMENTATION

13.4 Counterintuitive implementation

Over the course of 13 chapters we have seen numerous cases in which the implementation of a language feature may run counter to the programmer's intuition. Early reply—in which thread creation is usually delayed until the reply actually occurs—is but the most recent example. Others have included expression evaluation order (Section 6.1.4), subroutine in-lining (Section 9.2.4), tail recursion (Section 6.6.1), nonstack allocation of activation records (for unlimited extent—Section 3.6.2), out-of-order or even noncontiguous layout of record fields (Section 8.1.2), variable lookup in a central reference table (Section C-3.4.2), immutable objects under a reference model of variables (Section 6.1.2), and implementations of generics (Section 7.3.1) that share code among instances with different type parameters. A compiler may, particularly at higher levels of code improvement, produce code that differs dramatically from the form and organization of its input. Unless otherwise constrained by the language definition, an implementation is free to choose any translation that is provably equivalent to the input.

sult parameters depend. Drawing inspiration from the `detach` operation used to launch coroutines in Simula (Example 9.47), a few languages (SR and Hermes [SBG+91] among them) allow a callee to execute a reply operation that returns results to the caller *without* terminating. After an early reply, the two threads continue concurrently.

Semantically, the portion of the callee prior to the reply plays much the same role as the constructor of a Java or C# thread; the portion after the reply plays the role of the run method. The usual implementation is also similar, and may run counter to the programmer's intuition: since early reply is optional, and can appear in any subroutine, we can use the caller's thread to execute the initial portion of the callee, and create a new thread only when—and if—the callee replies instead of returning.

13.2.4 Implementation of Threads

As we noted near the beginning of Section 13.2, the threads of a concurrent program are usually implemented on top of one or more *processes* provided by the operating system. At one extreme, we could use a separate OS process for every thread; at the other extreme we could multiplex all of a program's threads on top of a single process. On a supercomputer with a separate core for every concurrent activity, or in a language in which threads are relatively heavyweight abstractions (long-lived, and created by the dozens rather than the thousands), the one-process-per-thread extreme is often acceptable. In a simple language on a uniprocessor, the all-threads-on-one-process extreme may be acceptable. Many language implementations adopt an intermediate approach, with a potentially very large number of threads running on top of some smaller but nontrivial number of processes (see Figure 13.6). ■

EXAMPLE 13.20
Multiplexing threads on processes

The problem with putting every thread on a separate process is that processes (even "lightweight" ones) are simply too expensive in many operating systems. Because they are implemented in the kernel, performing any operation on them requires a system call. Because they are general purpose, they provide features that most languages do not need, but have to pay for anyway. (Examples include separate address spaces, priorities, accounting information, and signal and I/O interfaces, all of which are beyond the scope of this book.) At the other extreme, there are two problems with putting all threads on top of a single process: first, it precludes parallel execution on a multicore or multiprocessor machine; second, if the currently running thread makes a system call that blocks (e.g., waiting for I/O), then none of the program's other threads can run, because the single process is suspended by the OS.

In the common two-level organization of concurrency (user-level threads on top of kernel-level processes), similar code appears at both levels of the system: the language run-time system implements threads on top of one or more processes in much the same way that the operating system implements processes on

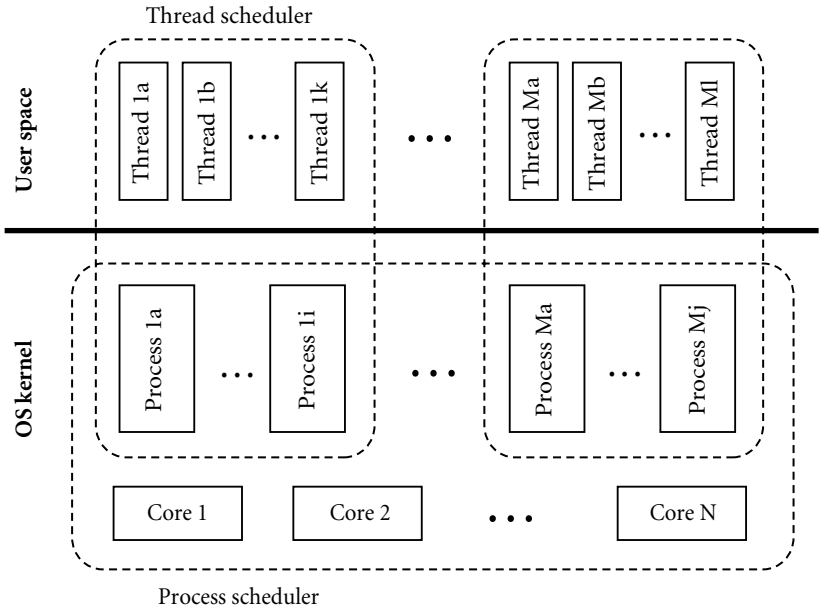


Figure 13.6 **Two-level implementation of threads.** A thread scheduler, implemented in a library or language run-time package, multiplexes threads on top of one or more kernel-level processes, just as the process scheduler, implemented in the operating system kernel, multiplexes processes on top of one or more physical cores.

top of one or more physical cores. We will use the terminology of threads on top of processes in the remainder of this section.

The typical implementation starts with coroutines (Section 9.5). Recall that coroutines are a sequential control-flow mechanism: the programmer can suspend the current coroutine and resume a specific alternative by calling the transfer operation. The argument to transfer is typically a pointer to the context block of the coroutine.

To turn coroutines into threads, we proceed in a series of three steps. First, we hide the argument to transfer by implementing a *scheduler* that chooses which thread to run next when the current thread yields the core. Second, we implement a *preemption* mechanism that suspends the current thread automatically on a regular basis, giving other threads a chance to run. Third, we allow the data structures that describe our collection of threads to be shared by more than one OS process, possibly on separate cores, so that threads can run on any of the processes.

Uniprocessor Scheduling

EXAMPLE 13.21
Cooperative multithreading on a uniprocessor

Figure 13.7 illustrates the data structures employed by a simple scheduler. At any particular time, a thread is either *blocked* (i.e., for synchronization) or *runnable*. A runnable thread may actually be running on some process or it may be awaiting

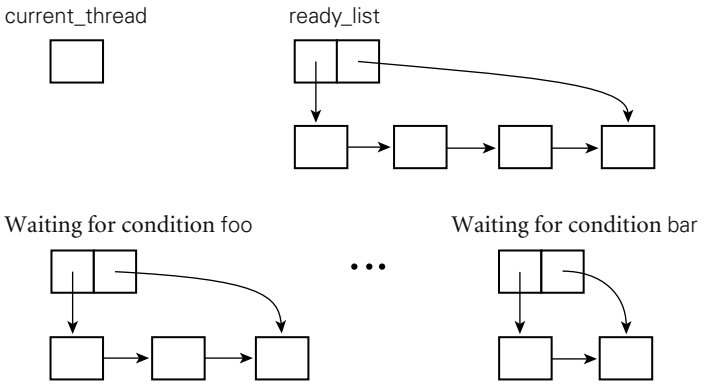


Figure 13.7 **Data structures of a simple scheduler.** A designated current_thread is running. Threads on the ready list are runnable. Other threads are blocked, waiting for various conditions to become true. If threads run on top of more than one OS-level process, each such process will have its own current_thread variable. If a thread makes a call into the operating system, its process may block in the kernel.

its chance to do so. Context blocks for threads that are runnable but not currently running reside on a queue called the *ready list*. Context blocks for threads that are blocked for scheduler-based synchronization reside in data structures (usually queues) associated with the conditions for which they are waiting. To yield the core to another thread, a running thread calls the scheduler:

```
procedure reschedule()
    t : thread := dequeue(ready_list)
    transfer(t)
```

Before calling into the scheduler, a thread that wants to run again at some point in the future must place its own context block in some appropriate data structure. If it is blocking for the sake of fairness—to give some other thread a chance to run—then it enqueues its context block on the ready list:

```
procedure yield()
    enqueue(ready_list, current_thread)
    reschedule()
```

To block for synchronization, a thread adds itself to a queue associated with the awaited condition:

```
procedure sleep_on(ref Q : queue of thread)
    enqueue(Q, current_thread)
    reschedule()
```

When a running thread performs an operation that makes a condition true, it removes one or more threads from the associated queue and enqueues them on the ready list.

Fairness becomes an issue whenever a thread may run for a significant amount of time while other threads are runnable. To give the illusion of concurrent activity, even on a uniprocessor, we need to make sure that each thread gets a frequent "slice" of the processor. With *cooperative multithreading*, any long-running thread must yield its core explicitly from time to time (e.g., at the tops of loops), to allow other threads to run. As noted in Section 13.1.1, this approach allows one improperly written thread to monopolize the system. Even with properly written threads, it leads to less than perfect fairness due to nonuniform times between yields in different threads.

Preemption

Ideally, we should like to multiplex each core fairly and at a relatively fine grain (i.e., many times per second) *without* requiring that threads call yield explicitly. On many systems we can do this in the language implementation by using timer signals for *preemptive multithreading*. When switching between threads we ask the operating system (which has access to the hardware clock) to deliver a signal to the currently running process at a specified time in the future. The OS delivers the signal by saving the context (registers and pc) of the process and transferring control to a previously specified *handler* routine in the language run-time system, as described in Section 9.6.1. When called, the handler modifies the state of the currently running thread to make it appear that the thread had just executed a call to the standard yield routine, and was about to execute its prologue. The handler then "returns" into yield, which transfers control to some other thread, as if the one that had been running had relinquished control of the process voluntarily.

EXAMPLE 13.22
A race condition in preemptive multithreading

Unfortunately, the fact that a signal may arrive at an arbitrary time introduces a race between voluntary calls to the scheduler and the automatic calls triggered by preemption. To illustrate the problem, suppose that a signal arrives when the currently running process has just enqueued the currently running thread onto the ready list in yield, and is about to call reschedule. When the signal handler "returns" into yield, the process will put the current thread into the ready list a second time. If at some point in the future the thread blocks for synchronization, its second entry in the ready list may cause it to run again immediately, when it should be waiting. Even worse problems can arise if a signal occurs in the middle of an enqueue, at a moment when the ready list is not even a properly structured queue. To resolve the race and avoid corruption of the ready list, thread packages commonly disable signal delivery during scheduler calls:

```
procedure yield()
    disable_signals()
    enqueue(ready_list, current_thread)
    reschedule()
    reenable_signals()
```

For this convention to work, *every* fragment of code that calls reschedule must disable signals prior to the call, and must reenable them afterward. (Recall that a similar mechanism served to protect data shared between the main program and

event handlers in Section 9.6.1.) In this case, because reschedule contains a call to transfer, signals may be disabled in one thread and reenabled in another. ■

EXAMPLE 13.23
Disabling signals during context switch

It turns out that the sleep_on routine must also assume that signals are disabled and enabled by the caller. To see why, suppose that a thread checks a condition, finds that it is false, and then calls sleep_on to suspend itself on a queue associated with the condition. Suppose further that a timer signal occurs immediately after checking the condition, but before the call to sleep_on. Finally, suppose that the thread that is allowed to run after the signal makes the condition true. Since the first thread never got a chance to put itself on the condition queue, the second thread will not find it to make it runnable. When the first thread runs again, it will immediately suspend itself, and may never be awakened. To close this *timing window*—this interval in which a concurrent event may compromise program correctness—the caller must ensure that signals are disabled before checking the condition:

```
disable_signals()
if not desired_condition
    sleep_on(condition_queue)
reenable_signals
```

On a uniprocessor, disabling signals allows the check and the sleep to occur as a single, *atomic* operation. ■

Multiprocessor Scheduling

We can extend our preemptive thread package to run on top of more than one OS-provided process by arranging for the processes to share the ready list and related data structures (condition queues, etc.; note that each process must have a *separate* current_thread variable). If the processes run on different physical cores, then more than one thread will be able to run at once. If the processes share a single core, then the program will be able to make forward progress even when all but one of the processes are blocked in the operating system. Any thread that is runnable is placed in the ready list, where it becomes a candidate for execution by any of the application's processes. When a process calls reschedule, the queue-based ready list we have been using in our examples will give it the longest-waiting thread. The ready list of a more elaborate scheduler might give priority to interactive or time-critical threads, or to threads that last ran on the current core, and may therefore still have data in the cache.

Just as preemption introduced a race between voluntary and automatic calls to scheduler operations, true or quasiparallelism introduces races between calls in separate OS processes. To resolve the races, we must implement additional synchronization to make scheduler operations in separate processes atomic. We will return to this subject in Section 13.3.4.

✓ CHECK YOUR UNDERSTANDING

11. Explain the differences among *coroutines*, *threads*, *lightweight processes*, and *heavyweight processes*.
12. What is *quasiparallelism*?
13. Describe the *bag of tasks* programming model.
14. What is *busy-waiting*? What is its principal alternative?
15. Name four explicitly concurrent programming languages.
16. Why don't message-passing programs require explicit synchronization mechanisms?
17. What are the tradeoffs between language-based and library-based implementations of concurrency?
18. Explain the difference between *data parallelism* and *task parallelism*.
19. Describe six different mechanisms commonly used to create new threads of control in a concurrent program.
20. In what sense is `fork/join` more powerful than *co-begin*?
21. What is a *thread pool* in Java? What purpose does it serve?
22. What is meant by a *two-level* thread implementation?
23. What is a *ready list*?
24. Describe the progressive implementation of scheduling, preemption, and (true) parallelism on top of coroutines.

13.3 Implementing Synchronization

As noted in Section 13.2.1, synchronization is the principal semantic challenge for shared-memory concurrent programs. Typically, synchronization serves either to make some operation *atomic* or to delay that operation until some necessary precondition holds. As noted in Section 13.1, atomicity is most commonly achieved with *mutual exclusion locks*. Mutual exclusion ensures that only one thread is executing some *critical section* of code at a given point in time. Critical sections typically transform a shared data structure from one consistent state to another.

Condition synchronization allows a thread to wait for a precondition, often expressed as a predicate on the value(s) in one or more shared variables. It is tempting to think of mutual exclusion as a form of condition synchronization (don't proceed until no other thread is in its critical section), but this sort of condition would require *consensus* among all extant threads, something that condition synchronization doesn't generally provide.

Our implementation of parallel threads, sketched at the end of Section 13.2.4, requires both atomicity and condition synchronization. Atomicity of operations on the ready list and related data structures ensures that they always satisfy a set of logical invariants: the lists are well formed, each thread is either running or resides in exactly one list, and so forth. Condition synchronization appears in the requirement that a process in need of a thread to run must wait until the ready list is nonempty.

It is worth emphasizing that we do not in general want to overly synchronize programs. To do so would eliminate opportunities for parallelism, which we generally want to maximize in the interest of performance. Moreover not all races are bad. If two processes are racing to dequeue the last thread from the ready list, we don't generally care which succeeds and which waits for another thread. We *do* care that the implementation of dequeue does not have *internal*, instruction-level races that might compromise the ready list's integrity. In general, our goal is to provide only as much synchronization as is necessary to eliminate "bad" races—those that might otherwise cause the program to produce incorrect results.

In the first subsection below we consider busy-wait synchronization. In the second we present an alternative, called *nonblocking synchronization*, in which atomicity is achieved without the need for mutual exclusion. In the third subsection we return to the subject of memory consistency (originally mentioned in Section 13.1.2), and discuss its implications for the semantics and implementation of language-level synchronization mechanisms. Finally, in Sections 13.3.4 and 13.3.5, we use busy-waiting among processes to implement a parallelism-safe thread scheduler, and then use this scheduler in turn to implement the most basic scheduler-based synchronization mechanism: namely, semaphores.

13.3.1 Busy-Wait Synchronization

Busy-wait condition synchronization is easy if we can cast a condition in the form of "location *X* contains value *Y*": a thread that needs to wait for the condition can simply read *X* in a loop, waiting for *Y* to appear. To wait for a condition involving more than one location, one needs atomicity to read the locations together, but given that, the implementation is again a simple loop.

Other forms of busy-wait synchronization are somewhat trickier. In the remainder of this section we consider *spin locks*, which provide mutual exclusion, and *barriers*, which ensure that no thread continues past a given point in a program until all threads have reached that point.

Spin Locks

Dekker is generally credited with finding the first two-thread mutual exclusion algorithm that requires no atomic instructions other than load and store. Dijkstra [Dij65] published a version that works for *n* threads in 1965. Peterson [Pet81] published a much simpler two-thread algorithm in 1981. Building on

```
type lock = Boolean := false;

procedure acquire_lock(ref L : lock)
    while not test_and_set(L)
        while L
            -- nothing -- spin
procedure release_lock(ref L : lock)
    L := false
```

Figure 13.8 **A simple test-and-`test_and_set` lock.** Waiting processes spin with ordinary read (`load`) instructions until the lock appears to be free, then use `test_and_set` to acquire it. The very first access is a `test_and_set`, for speed in the common (no competition) case.

Peterson's algorithm, one can construct a hierarchical n-thread lock, but it requires $O(n \log n)$ space and $O(\log n)$ time to get one thread into its critical section [YA93]. Lamport [Lam87][2] published an n-thread algorithm in 1987 that takes $O(n)$ space and $O(1)$ time in the absence of competition for the lock. Unfortunately, it requires $O(n)$ time when multiple threads attempt to enter their critical section at once.

While all of these algorithms are historically important, a practical spin lock needs to run in constant time and space, and for this one needs an atomic instruction that does more than load or store. Beginning in the 1960s, hardware designers began to equip their processors with instructions that read, modify, and write a memory location as a single atomic operation. The simplest such instruction is known as `test_and_set`. It sets a Boolean variable to `true` and returns an indication of whether the variable was previously `false`. Given `test_and_set`, acquiring a spin lock is almost trivial:

EXAMPLE 13.24
The basic test_and_set lock

```
while not test_and_set(L)
    -- nothing -- spin
```

In practice, embedding `test_and_set` in a loop tends to result in unacceptable amounts of communication on a multicore or multiprocessor machine, as the cache coherence mechanism attempts to reconcile writes by multiple cores attempting to acquire the lock. This overdemand for hardware resources is known as *contention*, and is a major obstacle to good performance on large machines.

EXAMPLE 13.25
Test-and-test_and_set

To reduce contention, the writers of synchronization libraries often employ a test-and-`test_and_set` lock, which spins with ordinary reads (satisfied by the cache) until it appears that the lock is free (see Figure 13.8). When a thread releases a lock there still tends to be a flurry of bus or interconnect activity as waiting

2 Leslie Lamport (1941–) has made a variety of seminal contributions to the theory of parallel and distributed computing, including synchronization algorithms, the notion of "happens-before" causality, Byzantine agreement, the Paxos consensus algorithm, and the temporal logic of actions. He also created the LaTeX macro package, with which this book was typeset. He received the ACM Turing Award in 2013.

threads perform their `test_and_sets`, but at least this activity happens only at the boundaries of critical sections. On a large machine, contention can be further reduced by implementing a *backoff* strategy, in which a thread that is unsuccessful in attempting to acquire a lock waits for a while before trying again.

Many processors provide atomic instructions more powerful than `test_and_set`. Some can swap the contents of a register and a memory location atomically. Some can add a constant to a memory location atomically, returning the previous value. Several processors, including the x86, the IA-64, and the SPARC, provide a particularly useful instruction called `compare_and_swap` (`CAS`). This instruction takes three arguments: a location, an expected value, and a new value. It checks to see whether the expected value appears in the specified location, and if so replaces it with the new value, atomically. In either case, it returns an indication of whether the change was made. Using instructions like `atomic_add` or `compare_and_swap`, one can build spin locks that are *fair*, in the sense that threads are guaranteed to acquire the lock in the order in which they first attempt to do so. One can also build locks that work well—with no contention, even at release time—on arbitrarily large machines [MCS91, Sco13]. These topics are beyond the scope of the current text. (It is perhaps worth mentioning that fairness is a two-edged sword: while it may be desirable from a semantic point of view, it tends to undermine cache locality, and interacts very badly with preemption.)

An important variant on mutual exclusion is the *reader–writer lock* [CHP71]. Reader–writer locks recognize that if several threads wish to *read* the same data structure, they can do so simultaneously without mutual interference. It is only when a thread wants to *write* the data structure that we need to prevent other threads from reading or writing simultaneously. Most busy-wait mutual exclusion locks can be extended to allow concurrent access by readers (see Exercise 13.8).

Barriers

Data-parallel algorithms are often structured as a series of high-level steps, or *phases*, typically expressed as iterations of some outermost loop. Correctness often depends on making sure that every thread completes the previous step before any moves on to the next. A *barrier* serves to provide this synchronization.

EXAMPLE 13.26
Barriers in finite element analysis

As a concrete example, *finite element analysis* models a physical object—a bridge, let us say—as an enormous collection of tiny fragments. Each fragment of the bridge imparts forces to the fragments adjacent to it. Gravity exerts a downward force on all fragments. Abutments exert an upward force on the fragments that make up base plates. The wind exerts forces on surface fragments. To evaluate stress on the bridge as a whole (e.g., to assess its stability and resistance to failures), a finite element program might divide the fragments among a large collection of threads (probably one per core). Beginning with the external forces, the program would then proceed through a sequence of iterations. In each iteration, each thread would recompute the forces on its fragments based on the forces found in the previous iteration. Between iterations, the threads would

```
shared count : integer := n
shared sense : Boolean := true
per-thread private local_sense : Boolean := true

procedure central_barrier()
    local_sense := not local_sense
        -- each thread toggles its own sense
    if fetch_and_decrement(count) = 1
        -- last arriving thread
        count := n                  -- reinitialize for next iteration
        sense := local_sense        -- allow other threads to proceed
    else
        repeat
            -- spin
        until sense = local_sense
```

Figure 13.9 A simple "sense-reversing" barrier. Each thread has its own copy of local_sense. Threads share a single copy of count and sense.

synchronize with a barrier. The program would halt when no thread found a significant change in any forces during the last iteration. ■

EXAMPLE 13.27
The "sense-reversing" barrier

The simplest way to implement a busy-wait barrier is to use a globally shared counter, modified by an atomic `fetch_and_decrement` instruction. The counter begins at n, the number of threads in the program. As each thread reaches the barrier it decrements the counter. If it is not the last to arrive, the thread then spins on a Boolean flag. The final thread (the one that changes the counter from 1 to 0) flips the Boolean flag, allowing the other threads to proceed. To make it easy to reuse the barrier data structures in successive iterations (known as barrier *episodes*), threads wait for alternating values of the flag each time through. Code for this simple barrier appears in Figure 13.9. ■

Like a simple spin lock, the "sense-reversing" barrier can lead to unacceptable levels of contention on large machines. Moreover the serialization of access to the counter implies that the time to achieve an n-thread barrier is $O(n)$. It is possible to do better, but even the fastest software barriers require $O(\log n)$ time to synchronize n threads [MCS91]. Large multiprocessors sometimes provide special hardware to reduce this bound to close to constant time.

EXAMPLE 13.28
Java 7 phasers

The Java 7 `Phaser` class provides unusually flexible barrier support. The set of participating threads can change from one phaser episode to another. When the number is large, the phaser can be *tiered* to run in logarithmic time. Moreover, arrival and departure can be specified as separate operations: in between, a thread can do work that (a) does not require that all other threads have arrived, and (b) does not have to be completed before any other threads depart. ■

13.3.2 Nonblocking Algorithms

When a lock is acquired at the beginning of a critical section, and released at the end, no other thread can execute a similarly protected piece of code at the same time. As long as every thread follows the same conventions, code within the critical section is atomic—it appears to happen all at once. But this is not the only possible way to achieve atomicity. Suppose we wish to make an arbitrary update to a shared location:

EXAMPLE 13.29
Atomic update with CAS

```
x := foo(x);
```

Note that this update involves at least two accesses to x: one to read the old value and one to write the new. We could protect the sequence with a lock:

```
acquire(L)
    r1 := x
    r2 := foo(r1)          -- probably a multi-instruction sequence
    x := r2
release(L)
```

But we can also do this without a lock, using `compare_and_swap`:

```
start:
    r1 := x
    r2 := foo(r1)          -- probably a multi-instruction sequence
    r2 := CAS(x, r1, r2)   -- replace x if it hasn't changed
    if !r2 goto start
```

If several cores execute this code simultaneously, one of them is guaranteed to succeed the first time around the loop. The others will fail and try again. This example illustrates that `CAS` is a *universal* primitive for single-location atomic update. A similar primitive, known as `load_linked/store_conditional`, is available on ARM, MIPS, and Power processors; we consider it in Exercise 13.7. ■

In our discussions thus far, we have used a definition of "blocking" that comes from operating systems: a thread that blocks gives up the core instead of actively spinning. An alternative definition comes from the theory of concurrent algorithms. Here the choice between spinning and giving up the core is immaterial: a thread is said to be "blocked" if it cannot make forward progress without action by other threads. Conversely, an operation is said to be *nonblocking* if in every reachable state of the system, any thread executing that operation is guaranteed to complete in a finite number of steps if it gets to run by itself (without further interference by other threads).

In this theoretical sense of the word, locks are inherently blocking, regardless of implementation: if one thread holds a lock, no other thread that needs that lock can proceed. By contrast, the `CAS`-based code of Example 13.29 is *nonblocking*: if the `CAS` operation fails, it is because some other thread has made progress.

Moreover if all threads but one stop running (e.g., because of preemption), the remaining thread is guaranteed to make progress.

We can generalize from Example 13.29 to design a variety of special-purpose concurrent data structures that operate without locks. Modifications to these structures often (though not always) follow the pattern

```
repeat
    prepare
    CAS                 -- (or some other atomic operation)
until success
clean up
```

If it reads more than one location, the "prepare" part of the algorithm may need to double-check to make sure that none of the values has changed (i.e., that all were read consistently) before moving on to the CAS. A read-only operation may simply return once this double-checking is successful.

In the `CAS`-based update of Example 13.29, the "prepare" part of the algorithm reads the old value of `x` and figures out what the new value ought to be; the "clean up" part is empty. In other algorithms there may be significant cleanup. In all cases, the keys to correctness are that (1) the "prepare" part is harmless if we need to repeat; (2) the `CAS`, if successful, logically completes the operation in a way that is visible to all other threads; and (3) the "clean up," if needed, can be performed by any thread if the original thread is delayed. Performing cleanup for another thread's operation is often referred to as *helping*.

EXAMPLE 13.30
The M&S queue

Figure 13.10 illustrates a widely used nonblocking concurrent queue. The `dequeue` operation does not require cleanup, but the `enqueue` operation does. To add an element to the end of the queue, a thread reads the current `tail` pointer to find the last node in the queue, and uses a `CAS` to change the `next` pointer of that node to point to the new node instead of being null. If the `CAS` succeeds (no other thread has already updated the relevant `next` pointer), then the new node has been inserted. As cleanup, the `tail` pointer must be updated to point to the new node, but any thread can do this—and will, if it discovers that `tail->next` is not null. ■

Nonblocking algorithms have several advantages over blocking algorithms. They are inherently tolerant of page faults and preemption: if a thread stops running partway through an operation, it never prevents other threads from making progress. Nonblocking algorithms can also safely be used in signal (event) and interrupt handlers, avoiding problems like the one described in Example 13.22. For several important data structures and algorithms, including stacks, queues, sorted lists, priority queues, hash tables, and memory management, nonblocking algorithms can also be faster than locks. Unfortunately, these algorithms tend to be exceptionally subtle and difficult to devise. They are used primarily in the implementation of language-level concurrency mechanisms and in standard library packages.

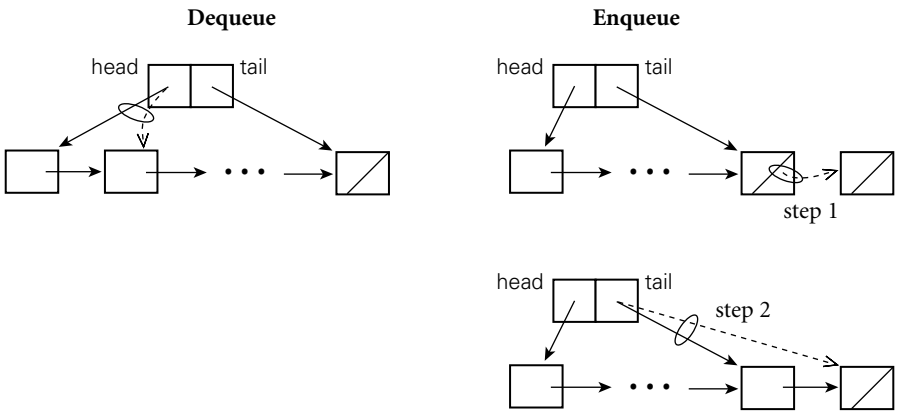


Figure 13.10 **Operations on a nonblocking concurrent queue.** In the dequeue operation (*left*), a single `CAS` swings the head pointer to the next node in the queue. In the enqueue operation (*right*), a first `CAS` changes the next pointer of the tail node to point at the new node, at which point the operation is said to have logically completed. A subsequent "cleanup" `CAS`, which can be performed by any thread, swings the tail pointer to point at the new node as well.

13.3.3 Memory Consistency

In all our discussions so far, we have depended, implicitly, on hardware memory coherence. Unfortunately, coherence alone is not enough to make a multiprocessor or even a single multicore processor behave as most programmers would expect. We must also worry, when more than one location is written at about the same time, about the *order* in which the writes become visible to different cores.

Intuitively, most programmers expect shared memory to be *sequentially consistent*—to make all writes visible to all cores in the same order, and to make any given core's writes visible in the order they were performed. Unfortunately, this behavior turns out to be very hard to implement efficiently—hard enough that most hardware designers simply don't provide it. Instead, they provide one of several *relaxed memory models*, in which certain loads and stores may appear to occur "out of order." Relaxed consistency has important ramifications for language designers, compiler writers, and the implementors of synchronization mechanisms and nonblocking algorithms.

The Cost of Ordering

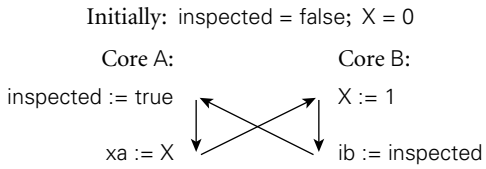
The fundamental problem with sequential consistency is that straightforward implementations require both hardware and compilers to serialize operations that we would rather be able to perform in arbitrary order.

EXAMPLE 13.31
Write buffers and consistency

Consider, for example, the implementation of an ordinary `store` instruction. In the event of a cache miss, this instruction can take hundreds of cycles to complete. Rather than wait, most processors are designed to continue executing

subsequent instructions while the store completes "in the background." Stores that are not yet visible in even the L1 cache (or that occurred after a store that is not yet visible) are kept in a queue called the *write buffer*. Loads are checked against the entries in this buffer, so a core always sees its own previous stores, and sequential programs execute correctly.

But consider a concurrent program in which thread A sets a flag (call it inspected) to true and then reads location X. At roughly the same time, thread B updates X from 0 to 1 and then reads the flag. If B's read reveals that inspected has not yet been set, the programmer might naturally assume that A is going to read new value (1) for X: after all, B updates X before checking the flag, and A sets the flag before reading X, so A cannot have read X already. On most machines, however, A can read X while its write of inspected is still in its write buffer. Likewise, B can read inspected while its write of X is still in its write buffer. The result can be very unintuitive behavior:



A's write of inspected precedes its read of X in program order. B's write of X precedes its read of inspected in program order. B's read of inspected appears to precede A's write of inspected, because it sees the unset value. And yet A's read of X appears to precede B's write of X as well, leaving us with xA = 0 and ib = false.

This sort of "temporal loop" may also be caused by standard compiler optimizations. Traditionally, a compiler is free to reorder instructions (in the absence of a data dependence) to improve the expected performance of the processor pipelines. In this example, a compiler that generates code for either A or B (without considering the other) may choose to reverse the order of operations on inspected and X, producing an apparent temporal loop even in the absence of hardware reordering. ■

Forcing Order with Fences and Synchronization Instructions

To avoid temporal loops, implementors of concurrent languages and libraries must generally use special *synchronization* or *memory fence* instructions. At some expense, these force orderings not normally guaranteed by the hardware.[3] Their presence also inhibits instruction reordering in the compiler.

In Example 13.31, both A and B must prevent their read from *bypassing* (completing before) the logically earlier write. Typically this can be accomplished by

---

3 Fences are also sometimes known as *memory barriers*. They are unrelated to the garbage collection barriers of Section 8.5.3 ("Tracing Collection"), the synchronization barriers of Section 13.3.1, or the RTL barriers of Section C-15.2.1.

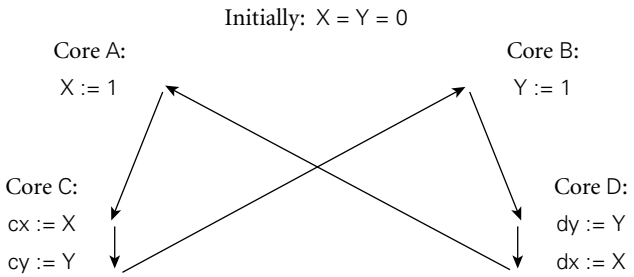


Figure 13.11 **Concurrent propagation of writes.** On some machines, it is possible for concurrent writes to reach cores in different orders. Arrows show apparent temporal ordering. Here C may read cy = 0 and cx = 1, while D reads dx = 0 and dy = 1.

identifying either the read or the write as a synchronization instruction (e.g., by implementing it with an `XCHG` instruction on the x86) or by inserting a fence between them (e.g., `membar StoreLoad` on the SPARC).

EXAMPLE 13.32
Distributed consistency

Sometimes, as in Example 13.31, the use of synchronization or fence instructions is enough to restore intuitive behavior. Other cases, however, require more significant changes to the program. An example appears in Figure 13.11. Cores A and B write locations X and Y, respectively. Both locations are read by cores C and D. If C is physically close to A in a distributed memory machine, and D is close to B, and if coherence messages propagate concurrently, we must consider the possibility that C and D will see the writes in opposite orders, leading to another temporal loop.

On machines where this problem arises, fences and synchronization instructions may not suffice to solve the problem. The language or library implementor (or even the application programmer) may need to bracket the writes of X and Y with (fenced) writes to some common location, to ensure that one of the original writes completes before the other starts. The most straightforward way to do this is to enclose the writes in a lock-based critical section. Even then, additional measures may be needed to ensure that the reads of X and Y are not executed out of order by either C or D. ■

Simplifying Language-Level Reasoning

For programs running on a single core, regardless of the complexity of the underlying pipeline and memory hierarchy, every manufacturer guarantees that instructions will appear to occur in *program order*: no instruction will fail to see the effects of some prior instruction, nor will it see the effects of any subsequent instruction. For programs running on a muilticore or multiprocessor machine, manufacturers also guarantee, under certain circumstances, that instructions executed on one core will be seen, in order, by instructions on other cores. Unfortunately, these circumstances vary from one machine to another. Some implementations of the MIPS and PA-RISC processors were sequentially consistent, as are IBM's z-Series mainframe machines: if a load on core B sees a value written

by a store on core A, then, transitively, everything before the store on A is guaranteed to have happened before everything after the load on B. Other processors and implementations are more relaxed. In particular, most machines admit the loop of Example 13.31. The SPARC and x86 preclude the loop of Example 13.32 (Figure 13.11), but the Power, ARM, and Itanium all allow it.

Given this variation across machines, what is a language designer to do? The answer, first suggested by Sarita Adve and now embedded in Java, C++, and (less formally) other languages, is to define a *memory model* that captures the notion of a "properly synchronized" program, and then provide the illusion of sequential consistency for all such programs. In effect, the memory model constitutes a contract between the programmer and the language implementation: if the programmer follows the rules of the contract, the implementation will hide all the ordering eccentricities of the underlying hardware.

In the usual formulation, memory models distinguish between "ordinary" variable accesses and special *synchronization accesses*; the latter include not only lock acquire and release, but also reads and writes of any variable declared with a special synchronization keyword (`volatile` in Java or C#, `atomic` in C++). Ordering of operations across cores is based solely on synchronization accesses. We say that operation A *happens before* operation C ($A \prec_{HB} C$) if (1) A comes before C in program order in a single thread; (2) A and C are synchronization operations and the language definition says that A comes before C; or (3) there exists an operation B such that $A \prec_{HB} B$ and $B \prec_{HB} C$.

Two ordinary accesses are said to *conflict* if they occur in different threads, they refer to the same location, and at least one of them is a write. They are said to constitute a *data race* if they conflict and they are not ordered—the implementation might allow either one to happen first, and program behavior might change as a result. Given these definitions, the memory model contract is straightforward: *executions of data-race–free programs are always sequentially consistent*.

In most cases, an acquire of a mutual exclusion lock is ordered after the most recent prior release. A read of a `volatile` (`atomic`) variable is ordered after the write that stored the value that was read. Various other operations (e.g., the *transactions* we will study in Section 13.4.4) may also contribute to cross-thread ordering.

EXAMPLE 13.33

Using `volatile` to avoid a data race

A simple example of ordering appears in Figure 13.12, where thread A sets variable initialized to indicate that it is safe for thread B to use reference p. If initialized had not been declared as volatile, there would be no cross-thread ordering between A's write of true and B's loop-terminating read. Access to both initialized and p would then be data races. Under the hood, the compiler would have been free to move the write of true before the initialization of p (remember, threads are often separately compiled, and the compiler has no obvious way to tell that the writes to p and initialized have anything to do with one another). Similarly, on a machine with a relaxed hardware memory model, the processor and memory system would have been free to perform the writes in either order, regardless of the order specified by the compiler in machine code. On B's core, it would also have been possible for either the compiler or the hardware to read p

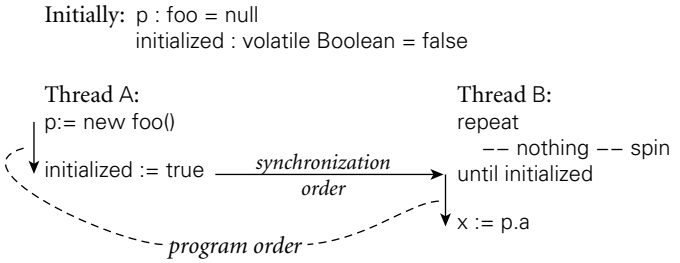


Figure 13.12 Protecting initialization with a volatile flag. Here labeling initialized as volatile avoids a data race, and ensures that B will not access p until it is safe to do so.

before confirming that initialized was true. The volatile declaration precludes all these undesirable possibilities.

Returning to Example 13.31, we might avoid a temporal loop by declaring both X and inspected as `volatile`, or by enclosing accesses to them in atomic operations, bracketed by lock acquire and release. In Example 13.32, `volatile` declarations on X and Y will again suffice to ensure sequential consistency, but the cost may be somewhat higher: on some machines, the compiler may need to use extra locks or special instructions to force a total order among writes to disjoint locations. ■

Synchronization races are common in multithreaded programs. Whether they are bugs or expected behavior depends on the application. Data races, on the other hand, are almost always program bugs. They are so hard to reason about—and so rarely useful—that the C++ memory model outlaws them altogether: given a program with a data race, a C++ implementation is permitted to display *any behavior whatsoever*. Ada has similar rules. For Java, by contrast, an emphasis on embedded applications motivated the language designers to constrain the behavior of racy programs in ways that would preserve the integrity of the underlying virtual machine. A Java program that contains a data race must continue to follow all the normal language rules, and any read that is not ordered with respect to a unique preceding write must return a value that might have been written by *some* previous write to the same location, or by a write that is unordered with respect to the read. We will return to the Java Memory Model in Section 13.4.3, after we have discussed the language's synchronization mechanisms.

13.3.4 Scheduler Implementation

EXAMPLE 13.34
Scheduling threads on processes

To implement user-level threads, OS-level processes must synchronize access to the ready list and condition queues, generally by means of spinning. Code for a simple *reentrant* thread scheduler (one that can be "reentered" safely by a second process before the first one has returned) appears in Figure 13.13. As in the code in Section 13.2.4, we disable timer signals before entering scheduler code, to

```
shared scheduler_lock : low_level_lock
shared ready_list : queue of thread
per-process private current_thread : thread

procedure reschedule()
    -- assume that scheduler_lock is already held and that timer signals are disabled
    t : thread
    loop
        t := dequeue(ready_list)
        if t ≠ null
            exit
        -- else wait for a thread to become runnable
        release_lock(scheduler_lock)
        -- window allows another thread to access ready_list (no point in reenabling
        -- signals; we're already trying to switch to a different thread)
        acquire_lock(scheduler_lock)
    transfer(t)
    -- caller must release scheduler_lock and reenable timer signals after we return

procedure yield()
    disable_signals()
    acquire_lock(scheduler_lock)
    enqueue(ready_list, current_thread)
    reschedule()
    release_lock(scheduler_lock)
    reenable_signals()

procedure sleep_on(ref Q : queue of thread)
    -- assume that caller has already disabled timer signals and acquired
    -- scheduler_lock, and will reverse these actions when we return
    enqueue(Q, current_thread)
    reschedule()
```

Figure 13.13 Pseudocode for part of a simple reentrant (parallelism-safe) scheduler. Every process has its own copy of current_thread. There is a single shared scheduler_lock and a single ready_list. If processes have dedicated cores, then the low_level_lock can be an ordinary spin lock; otherwise it can be a "spin-then-yield" lock (Figure 13.14). The loop inside reschedule busy-waits until the ready list is nonempty. The code for sleep_on cannot disable timer signals and acquire the scheduler lock itself, because the caller needs to test a condition and then block as a single atomic operation.

protect the ready list and condition queues from concurrent access by a process and its own signal handler. ■

EXAMPLE 13.35

A race condition in thread scheduling

Our code assumes a single "low-level" lock (scheduler_lock) that protects the entire scheduler. Before saving its context block on a queue (e.g., in yield or sleep_on), a thread must acquire the scheduler lock. It must then release the lock after returning from reschedule. Of course, because reschedule calls transfer, the lock will usually be acquired by one thread (the same one that disables timer

signals) and released by another (the same one that reenables timer signals). The code for yield can implement synchronization itself, because its work is self-contained. The code for sleep_on, on the other hand, cannot, because a thread must generally check a condition and block if necessary as a single atomic operation:

```
disable_signals()
acquire_lock(scheduler_lock)
if not desired_condition
    sleep_on(condition_queue)
release_lock(scheduler_lock)
reenable_signals()
```

If the signal and lock operations were moved inside of sleep_on, the following race could arise: thread *A* checks the condition and finds it to be false; thread *B* makes the condition true, but finds the condition queue to be empty; thread *A* sleeps on the condition queue forever. ■

A spin lock will suffice for the "low-level" lock that protects the ready list and condition queues, so long as every process runs on a different core. As we noted in Section 13.2.1, however, it makes little sense to spin for a condition that can only be made true by some other process using the core on which we are spinning. If we know that we're running on a uniprocessor, then we don't need a lock on the scheduler (just the disabled signals). If we *might* be running on a uniprocessor, however, or on a multiprocessor with fewer cores than processes, then we must be prepared to give up the core if unable to obtain a lock. The easiest way to do this is with a "spin-then-yield" lock, first suggested by Ousterhout [Ous82]. A simple example of such a lock appears in Figure 13.14. On a multiprogrammed machine, it might also be desirable to relinquish the core inside reschedule when the ready list is empty: though no other process of the current application will be able to do anything, overall system throughput may improve if we allow the operating system to give the core to a process from another application. ■

EXAMPLE 13.36
A "spin-then-yield" lock

On a large multiprocessor we might increase concurrency by employing a separate lock for each condition queue, and another for the ready list. We would have to be careful, however, to make sure it wasn't possible for one process to put a thread into a condition queue (or the ready list) and for another process to attempt to transfer into that thread before the first process had finished transferring out of it (see Exercise 13.13).

Scheduler-Based Synchronization

The problem with busy-wait synchronization is that it consumes processor cycles—cycles that are therefore unavailable for other computation. Busy-wait synchronization makes sense only if (1) one has nothing better to do with the current core, or (2) the expected wait time is less than the time that would be required to switch contexts to some other thread and then switch back again. To ensure acceptable performance on a wide variety of systems, most concurrent

```
type lock = Boolean := false;

procedure acquire_lock(ref L : lock)
    while not test_and_set(L)
        count := TIMEOUT
        while L
            count −:= 1
            if count = 0
                OS_yield()          −− relinquish core and drop priority
                count := TIMEOUT

procedure release_lock(ref L : lock)
    L := false
```

Figure 13.14 **A simple spin-then-yield lock, designed for execution on a multiprocessor that may be multiprogrammed (i.e., on which OS-level processes may be preempted).** If unable to acquire the lock in a fixed, short amount of time, a process calls the OS scheduler to yield its core and to lower its priority enough that other processes (if any) will be allowed to run. Hopefully the lock will be available the next time the yielding process is scheduled for execution.

programming languages employ scheduler-based synchronization mechanisms, which switch to a different thread when the one that was running blocks.

In the following subsection we consider *semaphores*, the most common form of scheduler-based synchronization. In Section 13.4 we consider the higher-level notions of monitors, conditional critical regions, and transactional memory. In each case, scheduler-based synchronization mechanisms remove the waiting thread from the scheduler's ready list, returning it only when the awaited condition is true (or is likely to be true). By contrast, a spin-then-yield lock is still a busy-wait mechanism: the currently running process relinquishes the core, but remains on the ready list. It will perform a `test_and_set` operation every time the lock appears to be free, until it finally succeeds. It is worth noting that busy-wait synchronization is generally "level-independent"—it can be thought of as synchronizing threads, processes, or cores, as desired. Scheduler-based synchronization is "level-*de*pendent"—it is specific to threads when implemented in the language run-time system, or to processes when implemented in the operating system.

EXAMPLE 13.37

The bounded buffer problem

We will use a *bounded buffer* abstraction to illustrate the semantics of various scheduler-based synchronization mechanisms. A bounded buffer is a concurrent queue of limited size into which *producer* threads insert data, and from which *consumer* threads remove data. The buffer serves to even out fluctuations in the relative rates of progress of the two classes of threads, increasing system throughput. A correct implementation of a bounded buffer requires both atomicity and condition synchronization: the former to ensure that no thread sees the buffer in an inconsistent state in the middle of some other thread's operation; the latter to force consumers to wait when the buffer is empty and producers to wait when the buffer is full. ■

```
type semaphore = record
    N : integer -- always non-negative
    Q : queue of threads

procedure P(ref S : semaphore)
    disable_signals()
    acquire_lock(scheduler_lock)
    if S.N > 0
        S.N -:= 1
    else
        sleep_on(S.Q)
    release_lock(scheduler_lock)
    reenable_signals()

procedure V(ref S : semaphore)
    disable_signals()
    acquire_lock(scheduler_lock)
    if S.Q is nonempty
        enqueue(ready_list, dequeue(S.Q))
    else
        S.N +:= 1
    release_lock(scheduler_lock)
    reenable_signals()
```

Figure 13.15 Semaphore operations, for use with the scheduler code of Figure 13.13.

13.3.5 Semaphores

Semaphores are the oldest of the scheduler-based synchronization mechanisms. They were described by Dijkstra in the mid-1960s [Dij68a], and appear in Algol 68. They are still heavily used today, particularly in library-based implementations of concurrency.

EXAMPLE 13.38
Semaphore implementation

A semaphore is basically a counter with two associated operations, P and V.[4] A thread that calls V atomically increments the counter. A thread that calls P waits until the counter is positive and then decrements it. We generally require that semaphores be fair, in the sense that threads complete P operations in the order that they started them. Implementations of P and V in terms of our scheduler operations appear in Figure 13.15. Note that we have elided the matching increment and decrement when a V allows a thread that is waiting in P to continue right away. ■

A semaphore whose counter is initialized to one and for which P and V operations always occur in matched pairs is known as a *binary semaphore*. It serves as

4 P stands for the Dutch word *passeren* (to pass) or *proberen* (to test); V stands for *vrijgeven* (to release) or *verhogen* (to increment). To keep them straight, speakers of English may wish to think of P as standing for "pause," since a thread will pause at a P operation if the semaphore count is negative. Algol 68 calls the P and V operations `down` and `up`, respectively.

```
shared buf : array [1..SIZE] of bdata
shared next_full, next_empty : integer := 1, 1
shared mutex : semaphore := 1
shared empty_slots, full_slots : semaphore := SIZE, 0

procedure insert(d : bdata)
    P(empty_slots)
    P(mutex)
    buf[next_empty] := d
    next_empty := next_empty mod SIZE + 1
    V(mutex)
    V(full_slots)

function remove() : bdata
    P(full_slots)
    P(mutex)
    d : bdata := buf[next_full]
    next_full := next_full mod SIZE + 1
    V(mutex)
    V(empty_slots)
    return d
```

Figure 13.16 **Semaphore-based code for a bounded buffer.** The mutex binary semaphore protects the data structure proper. The full_slots and empty_slots general semaphores ensure that no operation starts until it is safe to do so.

a scheduler-based mutual exclusion lock: the P operation acquires the lock; V releases it. More generally, a semaphore whose counter is initialized to k can be used to arbitrate access to k copies of some resource. The value of the counter at any particular time indicates the number of copies not currently in use. Exercise 13.19 notes that binary semaphores can be used to implement general semaphores, so the two are of equal expressive power, if not of equal convenience.

EXAMPLE 13.39
Bounded buffer with semaphores

Figure 13.16 shows a semaphore-based solution to the bounded buffer problem. It uses a binary semaphore for mutual exclusion, and two general (or *counting*) semaphores for condition synchronization. Exercise 13.17 considers the use of semaphores to construct an n-thread barrier. ■

CHECK YOUR UNDERSTANDING

25. What is *mutual exclusion*? What is a *critical section*?
26. What does it mean for an operation to be *atomic*? Explain the difference between atomicity and *condition synchronization*.
27. Describe the behavior of a `test_and_set` instruction. Show how to use it to build a *spin lock*.
28. Describe the behavior of the `compare_and_swap` instruction. What advantages does it offer in comparison to `test_and_set`?

29. Explain how a *reader–writer lock* differs from an "ordinary" lock.
30. What is a *barrier*? In what types of programs are barriers common?
31. What does it mean for an algorithm to be *nonblocking*? What advantages do nonblocking algorithms have over algorithms based on locks?
32. What is *sequential consistency*? Why is it difficult to implement?
33. What information is provided by a *memory consistency model*? What is the relationship between hardware-level and language-level memory models?
34. Explain how to extend a preemptive uniprocessor scheduler to work correctly on a multiprocessor.
35. What is a *spin-then-yield* lock?
36. What is a *bounded buffer*?
37. What is a *semaphore*? What operations does it support? How do *binary* and *general* semaphores differ?

13.4 Language-Level Constructs

Though widely used, semaphores are also widely considered to be too "low level" for well-structured, maintainable code. They suffer from two principal problems. First, because their operations are simply subroutine calls, it is easy to leave one out (e.g., on a control path with several nested `if` statements). Second, unless they are hidden inside an abstraction, uses of a given semaphore tend to get scattered throughout a program, making it difficult to track them down for purposes of software maintenance.

13.4.1 Monitors

Monitors were suggested by Dijkstra [Dij72] as a solution to the problems of semaphores. They were developed more thoroughly by Brinch Hansen [Bri73], and formalized by Hoare [Hoa74] in the early 1970s. They have been incorporated into at least a score of languages, of which Concurrent Pascal [Bri75], Modula (1) [Wir77b], and Mesa [LR80] were probably the most influential.[5] They

5 Together with Smalltalk and Interlisp, Mesa was one of three influential languages to emerge from Xerox's Palo Alto Research Center in the 1970s. All three were developed on the Alto personal computer, which pioneered such concepts as the bitmapped display, the mouse, the graphical user interface, WYSIWYG editing, Ethernet networking, and the laser printer. The Mesa project was led by Butler Lampson (1943–), who played a key role in the later development of Euclid and Cedar as well. For his contributions to personal and distributed computing, Lampson received the ACM Turing Award in 1992.

```
monitor bounded_buf
imports bdata, SIZE
exports insert, remove

    buf : array [1..SIZE] of bdata
    next_full, next_empty : integer := 1, 1
    full_slots : integer := 0
    full_slot, empty_slot : condition

    entry insert(d : bdata)
        if full_slots = SIZE
            wait(empty_slot)
        buf[next_empty] := d
        next_empty := next_empty mod SIZE + 1
        full_slots +:= 1
        signal(full_slot)

    entry remove() : bdata
        if full_slots = 0
            wait(full_slot)
        d : bdata := buf[next_full]
        next_full := next_full mod SIZE + 1
        full_slots −:= 1
        signal(empty_slot)
        return d
```

Figure 13.17 Monitor-based code for a bounded buffer. Insert and remove are *entry* subroutines: they require exclusive access to the monitor's data. Because conditions are memory-less, both insert and remove can safely end their operation with a signal.

also strongly influenced the design of Java's synchronization mechanisms, which we will consider in Section 13.4.3.

A monitor is a module or object with operations, internal state, and a number of *condition variables*. Only one operation of a given monitor is allowed to be active at a given point in time. A thread that calls a busy monitor is automatically delayed until the monitor is free. On behalf of its calling thread, any operation may suspend itself by *wait*ing on a condition variable. An operation may also *signal* a condition variable, in which case one of the waiting threads is resumed, usually the one that waited first.

Because the operations (*entries*) of a monitor automatically exclude one another in time, the programmer is relieved of the responsibility of using P and V operations correctly. Moreover because the monitor is an abstraction, all operations on the encapsulated data, including synchronization, are collected together in one place. Figure 13.17 shows a monitor-based solution to the bounded buffer problem. It is worth emphasizing that monitor condition variables are not the same as semaphores. Specifically, they have no "memory": if no thread is waiting on a condition at the time that a signal occurs, then the signal has no effect.

EXAMPLE 13.40
Bounded buffer monitor

By contrast a V operation on a semaphore increments the semaphore's counter, allowing some future P operation to succeed, even if none is waiting now. ■

Semantic Details

Hoare's definition of monitors employs one thread queue for every condition variable, plus two bookkeeping queues: the *entry queue* and the *urgent queue*. A thread that attempts to enter a busy monitor waits in the entry queue. When a thread executes a signal operation from within a monitor, and some other thread is waiting on the specified condition, then the signaling thread waits on the monitor's urgent queue and the first thread on the appropriate condition queue obtains control of the monitor. If no thread is waiting on the signaled condition, then the signal operation is a no-op. When a thread leaves a monitor, either by completing its operation or by waiting on a condition, it unblocks the first thread on the urgent queue or, if the urgent queue is empty, the first thread on the entry queue, if any.

Many monitor implementations dispense with the urgent queue, or make other changes to Hoare's original definition. From the programmer's point of view, the two principal areas of variation are the semantics of the signal operation and the management of mutual exclusion when a thread waits inside a nested sequence of two or more monitor calls. We will return to these issues below.

Correctness for monitors depends on maintaining a *monitor invariant*. The invariant is a predicate that captures the notion that "the state of the monitor is consistent." The invariant needs to be true initially, and at monitor exit. It also needs to be true at every wait statement and, in a Hoare monitor, at signal operations as well. For our bounded buffer example, a suitable invariant would assert that full_slots correctly indicates the number of items in the buffer, and that those items lie in slots numbered next_full through next_empty − 1 (mod SIZE). Careful inspection of the code in Figure 13.17 reveals that the invariant does indeed hold initially, and that any time we modify one of the variables mentioned in the invariant, we always modify the others accordingly before waiting, signaling, or returning from an entry.

Hoare defined his monitors in terms of semaphores. Conversely, it is easy to define semaphores in terms of monitors (Exercise 13.18). Together, the two definitions prove that semaphores and monitors are equally powerful: each can express all forms of synchronization expressible with the other.

Signals as Hints and Absolutes

In general, one signals a condition variable when some condition on which a thread may be waiting has become true. If we want to guarantee that the condition is still true when the thread wakes up, then we need to switch to the thread as soon as the signal occurs—hence the need for the urgent queue, and the need to ensure the monitor invariant at signal operations. In practice, switching contexts on a signal tends to induce unnecessary scheduling overhead: a signaling thread seldom changes the condition associated with the signal during the remainder of its operation. To reduce the overhead, and to eliminate the need to ensure the

EXAMPLE 13.41

How to wait for a signal (hint or absolute)

monitor invariant, Mesa specifies that signals are only *hints*: the language runtime system moves some waiting thread to the ready list, but the signaler retains control of the monitor, and the waiter must recheck the condition when it awakes. In effect, the standard idiom

```
if not desired_condition
    wait(condition_variable)
```

in a Hoare monitor becomes the following in Mesa:

```
while not desired_condition
    wait(condition_variable)
```

DESIGN & IMPLEMENTATION

13.5 Monitor signal semantics

By specifying that signals are hints, instead of absolutes, Mesa and Modula-3 (and similarly Java and C#, which we consider in Section 13.4.3) avoid the need to perform an immediate context switch from a signaler to a waiting thread. They also admit simpler, though less efficient implementations that lack a one-to-one correspondence between signals and thread queues, or that do not necessarily guarantee that a waiting thread will be the first to run in its monitor after the signal occurs. This approach can lead to complications, however, if we want to ensure that an appropriate thread always runs in the wake of a signal. Suppose an awakened thread rechecks its condition and discovers that it still can't run. If there may be some other thread that could run, the erroneously awakened thread may need to resignal the condition before it waits again:

```
if not desired_condition
    loop
        wait(condition_variable)
        if desired_condition
            break
        signal(condition_variable)
```

In effect the signal "cascades" from thread to thread until some thread is able to run. (If it is possible that *no* waiting thread will be able to run, then we will need additional logic to stop the cascading when every thread has been checked.) Alternatively, the thread that makes a condition (potentially) true can use a special broadcast version of the signal operation to awaken *all* waiting threads at once. Each thread will then recheck the condition and if appropriate wait again, without the need for explicit cascading. In either case (cascading signals or broadcast), signals as hints trade potentially high overhead in the worst case for potentially low overhead in the common case and a potentially simpler implementation.

Modula-3 takes a similar approach. An alternative appears in Concurrent Pascal, which specifies that a `signal` operation causes an immediate return from the monitor operation in which it appears. This rule keeps overhead low, and also preserves invariants, but precludes algorithms in which a thread does useful work in a monitor after `signal`ing a condition. ■

Nested Monitor Calls

In most monitor languages, a wait in a nested sequence of monitor operations will release mutual exclusion on the innermost monitor, but will leave the outer monitors locked. This situation can lead to *deadlock* if the only way for another thread to reach a corresponding signal operation is through the same outer monitor(s). In general, we use the term "deadlock" to describe any situation in which a collection of threads are all waiting for each other, and none of them can proceed. In this specific case, the thread that entered the outer monitor first is waiting for the second thread to execute a signal operation; the second thread, however, is waiting for the first to leave the monitor.

The alternative—to release exclusion on outer monitors when waiting in an inner one—was adopted by several early monitor implementations for uniprocessors, including the original implementation of Modula [Wir77a]. It has a significant semantic drawback, however: it requires that the monitor invariant hold not only at monitor exit and (perhaps) at signal operations, but also at any subroutine call that may result in a wait or (with Hoare semantics) a signal in a nested monitor. Such calls may not all be known to the programmer; they are certainly not syntactically distinguished in the source.

DESIGN & IMPLEMENTATION

13.6 The nested monitor problem

While maintaining exclusion on outer monitor(s) when waiting in an inner one may lead to deadlock with a signaling thread, releasing those outer monitors may lead to similar (if a bit more subtle) deadlocks. When a waiting thread awakens it must reacquire exclusion on both inner and outer monitors. The innermost monitor is of course available, because the matching signal happened there, but there is in general no way to ensure that unrelated threads will not be busy in the outer monitor(s). Moreover one of those threads may need access to the inner monitor in order to complete its work and release the outer monitor(s). If we insist that the awakened thread be the first to run in the inner monitor after the signal, then deadlock will result. One way to avoid this problem is to arrange for mutual exclusion across *all* the monitors of a program. This solution severely limits concurrency in multiprocessor implementations, but may be acceptable on a uniprocessor. A more general solution is addressed in Exercise 13.21.

13.4.2 Conditional Critical Regions

EXAMPLE 13.42
Original CCR syntax

Conditional critical regions (CCRs) are another alternative to semaphores, proposed by Brinch Hansen at about the same time as monitors [Bri73]. A critical region is a syntactically delimited critical section in which code is permitted to access a *protected* variable. A *conditional* critical region also specifies a Boolean condition, which must be true before control will enter the region:

```
region protected_variable, when Boolean_condition do
    ...
end region
```

No thread can access a protected variable except within a region statement for that variable, and any thread that reaches a region statement waits until the condition is true and no other thread is currently in a region for the same variable. Regions can nest, though as with nested monitor calls, the programmer needs to worry about deadlock. Figure 13.18 uses CCRs to implement a bounded buffer. ■

Conditional critical regions appeared in the concurrent language Edison [Bri81], and also seem to have influenced the synchronization mechanisms of Ada 95 and Java/C#. These later languages might be said to blend the features of monitors and CCRs, albeit in different ways.

Synchronization in Ada 95

The principal mechanism for synchronization in Ada, introduced in Ada 83, is based on message passing; we will describe it in Section 13.5. Ada 95 augments this mechanism with a notion of *protected object*. A protected object can have three types of methods: functions, procedures, and *entries*. Functions can only read the fields of the object; procedures and entries can read and write them. An

DESIGN & IMPLEMENTATION

13.7 Conditional critical regions

Conditional critical regions avoid the question of `signal` semantics, because they use explicit Boolean conditions instead of condition variables, and because conditions can be awaited only at the beginning of critical regions. At the same time, they introduce potentially significant inefficiency. In the general case, the code used to exit a conditional critical region must tentatively resume each waiting thread, allowing that thread to recheck its condition in its own referencing environment. Optimizations are possible in certain special cases (e.g., for conditions that depend only on global variables, or that consist of only a single Boolean variable), but in the worst case it may be necessary to perform context switches in and out of every waiting thread on every exit from a region.

```
buffer : record
    buf : array [1..SIZE] of bdata
    next_full, next_empty : integer := 1, 1
    full_slots : integer := 0

procedure insert(d : bdata)
    region buffer when full_slots < SIZE
        buf[next_empty] := d
        next_empty := next_empty mod SIZE + 1
        full_slots −:= 1

function remove() : bdata
    region buffer when full_slots > 0
        d : bdata := buf[next_full]
        next_full := next_full mod SIZE + 1
        full_slots +:= 1
    return d
```

Figure 13.18 **Conditional critical regions for a bounded buffer.** Boolean conditions on the region statements eliminate the need for explicit condition variables.

implicit reader–writer lock on the protected object ensures that potentially conflicting operations exclude one another in time: a procedure or entry obtains exclusive access to the object; a function can operate concurrently with other functions, but not with a procedure or entry.

Procedures and entries differ from one another in two important ways. First, an entry can have a Boolean expression *guard*, for which the calling task (thread) will wait before beginning execution (much as it would for the condition of a CCR). Second, an entry supports three special forms of call: *timed* calls, which abort after waiting for a specified amount of time, *conditional* calls, which execute alternative code if the call cannot proceed immediately, and *asynchronous* calls, which begin executing alternative code immediately, but abort it if the call is able to proceed before the alternative completes.

In comparison to the conditions of CCRs, the guards on entries of protected objects in Ada 95 admit a more efficient implementation, because they do not have to be evaluated in the context of the calling thread. Moreover, because all guards are gathered together in the definition of the protected object, the compiler can generate code to test them as a group as efficiently as possible, in a manner suggested by Kessels [Kes77]. Though an Ada task cannot wait on a condition in the middle of an entry (only at the beginning), it can *requeue* itself on another entry, achieving much the same effect. Ada 95 code for a bounded buffer would closely resemble the pseudocode of Figure 13.18; we leave the details to Exercise 13.23.

13.4.3 Synchronization in Java

EXAMPLE 13.43
`synchronized` statement in Java

In Java, every object accessible to more than one thread has an implicit mutual exclusion lock, acquired and released by means of `synchronized` statements:

```
synchronized (my_shared_obj) {
    ... // critical section
}
```

All executions of `synchronized` statements that refer to the same shared object exclude one another in time. `Synchronized` statements that refer to different objects may proceed concurrently. As a form of syntactic sugar, a method of a class may be prefixed with the `synchronized` keyword, in which case the body of the method is considered to have been surrounded by an implicit `synchronized (this)` statement. Invocations of nonsynchronized methods of a shared object—and direct accesses to public fields—can proceed concurrently with each other, or with a `synchronized` statement or method.

Within a `synchronized` statement or method, a thread can suspend itself by calling the predefined method `wait`. `Wait` has no arguments in Java: the core language does not distinguish among the different reasons why threads may be suspended on a given object (the `java.util.concurrent` library, which became standard with Java 5, does provide a mechanism for multiple conditions; more on this below). Like Mesa, Java allows a thread to be awoken for spurious reasons, or after a delay; programs must therefore embed the use of `wait` within a condition-testing loop:

EXAMPLE 13.44
`notify` as hint in Java

```
while (!condition) {
    wait();
}
```

A thread that calls the `wait` method of an object releases the object's lock. With nested `synchronized` statements, however, or with nested calls to `synchronized` methods, the thread does *not* release locks on any other objects.

To resume a thread that is suspended on a given object, some other thread must execute the predefined method `notify` from within a `synchronized` statement or method that refers to the same object. Like `wait`, `notify` has no arguments. In response to a `notify` call, the language run-time system picks an arbitrary thread suspended on the object and makes it runnable. If there are no such threads, then the `notify` is a no-op. As in Mesa, it may sometimes be appropriate to awaken *all* threads waiting in a given object; Java provides a built-in `notifyAll` method for this purpose.

If threads are waiting for more than one condition (i.e., if their `waits` are embedded in dissimilar loops), there is no guarantee that the "right" thread will awaken. To ensure that an appropriate thread does wake up, the programmer may choose to use `notifyAll` instead of `notify`. To ensure that only *one* thread continues after wakeup, the first thread to discover that its condition has been

satisfied must modify the state of the object in such a way that other awakened threads, when they get to run, will simply go back to sleep. Unfortunately, since all waiting threads will end up reevaluating their conditions every time one of them can run, this "solution" to the multiple-condition problem can be quite expensive.

The mechanisms for synchronization in C# are similar to the Java mechanisms just described. The C# `lock` statement is similar to Java's `synchronized`. It cannot be used to label a method, but a similar effect can be achieved (a bit more clumsily) by specifying a `Synchronized` *attribute* for the method. The methods `Pulse` and `PulseAll` are used instead of `notify` and `notifyAll`.

Lock Variables

In early versions of Java, programmers concerned with efficiency generally needed to devise algorithms in which threads were never waiting for more than one condition within a given object at a given time. The `java.util.concurrent` package, introduced in Java 5, provides a more general solution. (Similar solutions are possible in C#, but are not in the standard library.) As an alternative to `synchronized` statements and methods, modern Java programmers can create explicit `Lock` variables. Code that might once have been written

EXAMPLE 13.45
`Lock` variables in Java 5

```
synchronized (my_shared_obj) {
    ... // critical section
}
```

DESIGN & IMPLEMENTATION

13.8 Condition variables in Java

As illustrated by Mesa and Java, the distinction between monitors and CCRs is somewhat blurry. It turns out to be possible (see Exercise 13.22) to solve completely general synchronization problems in such a way that for every protected object there is only one Boolean condition on which threads ever spin. The solutions, however, may not be pretty: they amount to low-level use of semaphores, without the implicit mutual exclusion of *synchronized* statements and methods. For programs that are naturally expressed with multiple conditions, Java's basic synchronization mechanism (and the similar mechanism in C#) may force the programmer to choose between elegance and efficiency. The concurrency enhancements of Java 5 were a deliberate attempt to lessen this dilemma: `Lock` variables retain the distinction between mutual exclusion and condition synchronization characteristic of both monitors and CCRs, while allowing the programmer to partition waiting threads into equivalence classes that can be awoken independently. By varying the fineness of the partition the programmer can choose essentially any point on the spectrum between the simplicity of CCRs and the efficiency of Hoare-style monitors. Exercises 13.24 through 13.26 explore this issue further using bounded buffers as a running example.

may now be written

```
Lock l = new ReentrantLock();
l.lock();
try {
    ... // critical section
} finally {
    l.unlock();
}
```

A similar interface supports reader–writer locks. ■

Like semaphores, Java `Lock` variables lack the implicit release at end of scope associated with `synchronized` statements and methods. The need for an explicit release introduces a potential source of bugs, but allows programmers to create algorithms in which locks are acquired and released in non-LIFO order (see Example 13.14). In a manner reminiscent of the `timed` entry calls of Ada 95, Java `Lock` variables also support a `tryLock` method, which acquires the lock only if it is available immediately, or within an optionally specified timeout interval (a Boolean return value indicates whether the attempt was successful). Finally, a `Lock` variable may have an arbitrary number of associated `Condition` variables, making it easy to write algorithms in which threads wait for multiple conditions, without resorting to `notifyAll`:

EXAMPLE 13.46
Multiple `Conditions` in Java 5

```
Condition c1 = l.newCondition();
Condition c2 = l.newCondition();
...
c1.await();
...
c2.signal();
```

■

Java objects that use only `synchronized` methods (no locks or `synchronized` statements) closely resemble Mesa monitors in which there is a limit of one condition variable per monitor (and in fact objects with `synchronized` statements are sometimes referred to as monitors in Java). By the same token, a `synchronized` statement in Java that begins with a `wait` in a loop resembles a CCR in which the retesting of conditions has been made explicit. Because `notify` also is explicit, a Java implementation need not reevaluate conditions (or wake up threads that do so explicitly) on every exit from a critical section—only those in which a `notify` occurs.

The Java Memory Model

The Java Memory Model, which we introduced in Section 13.3.3, specifies exactly which operations are guaranteed to be ordered across threads. It also specifies, for every pair of reads and writes in a program execution, whether the read is permitted to return the value written by the write.

Informally, a Java thread is allowed to buffer or reorder its writes (in hardware or in software) until the point at which it writes a `volatile` variable or leaves a

monitor (releases a lock, leaves a `synchronized` block, or `waits`). At that point all its previous writes must be visible to other threads. Similarly, a thread is allowed to keep cached copies of values written by other threads until it reads a `volatile` variable or enters a monitor (acquires a lock, enters a `synchronized` block, or wakes up from a `wait`). At that point any subsequent reads must obtain new copies of anything that has been written by other threads.

The compiler is free to reorder ordinary reads and writes in the absence of intrathread data dependences. It can also move ordinary reads and writes down past a subsequent `volatile` read, up past a previous `volatile` write, or into a `synchronized` block from above or below. It cannot reorder `volatile` accesses, monitor entry, or monitor exit with respect to one another.

If the compiler can prove that a `volatile` variable or monitor isn't used by more than one thread during a given interval of time, it can reorder its operations like ordinary accesses. For data-race-free programs, these rules ensure the appearance of sequential consistency. Moreover even in the presence of races, Java implementations ensure that reads and writes of object references and of 32-bit and smaller quantities are always atomic, and that every read returns the value written either by some unordered write or by some immediately preceding ordered write.

Formalization of the Java memory model proved to be a surprisingly difficult task. Most of the difficulty stemmed from the desire to specify meaningful semantics for programs with data races. The C++11 memory model, also introduced in Section 13.3.3, avoids this complexity by simply prohibiting such programs. To first approximation, C++ defines a *happens-before* ordering on memory accesses, similar to the ordering in Java, and then guarantees sequential consistency for programs in which all conflicting accesses are ordered. Modest additional complexity is introduced by allowing the programmer to specify weaker ordering on individual reads and writes of `atomic` variables; we consider this feature in Exploration 13.42.

13.4.4 Transactional Memory

All the general-purpose mechanisms we have considered for atomicity—semaphores, monitors, conditional critical regions—are essentially syntactic variants on locks. Critical sections that need to exclude one another must acquire and release the same lock. Critical sections that are mutually independent can run in parallel only if they acquire and release separate locks. This creates an unfortunate tradeoff for programmers: it is easy to write a data-race-free program with a single lock, but such a program will not *scale*: as cores and threads are added, the lock will become a bottleneck, and program performance will stagnate. To increase scalability, skillful programmers partition their program data into equivalence classes, each protected by a separate lock. A critical section must then acquire the locks for every accessed equivalence class. If different critical sections acquire locks in different orders, deadlock can result. Enforcing a common order can be difficult, however, because we may not be able to predict, when an

operation starts, which data it will eventually need to access. Worse, the fact that correctness depends on locking order means that lock-based program fragments do not *compose*: we cannot take existing lock-based abstractions and safely call them from within a new critical section.

These issues suggest that locks may be too *low level* a mechanism. From a semantic point of view, the mapping between locks and critical sections is an implementation detail; all we really want is a composable atomic construct. Transactional memory (TM) is an attempt to provide exactly that.

Atomicity without Locks

Transactions have long been used, with great success, to achieve atomicity for database operations. The usual implementation is *speculative*: transactions in different threads proceed concurrently unless and until they *conflict* for access to some common record in the database. In the absence of conflicts, transactions run perfectly in parallel. When conflicts arise, the underlying system arbitrates between the conflicting threads. One gets to continue, and hopefully *commit* its updates to the database; the others *abort* and start over (after "rolling back" the work they had done so far). The overall effect is that transactions achieve significant parallelism at the implementation level, but appear to *serialize* in some global total order at the level of program semantics.

The idea of using more lightweight transactions to achieve atomicity for operations on in-memory data structures dates from 1993, when Herlihy and Moss proposed what was essentially a multiword generalization of the `load_linked/store_conditional`, instructions mentioned in Example 13.29. Their *transactional memory* (TM) began to receive renewed attention (and higher-level semantics) about a decade later, when it became clear to many researchers that multicore processors were going to be successful only with the development of simpler programming techniques.

EXAMPLE 13.47
A simple atomic block

The basic idea of TM is very simple: the programmer labels code blocks as atomic—

```
atomic {
    -- your code here
}
```

—and the underlying system takes responsibility for executing these blocks in parallel whenever possible. If the code inside the atomic block can safely be rolled back in the event of conflict, then the implementation can be based on speculation. ■

EXAMPLE 13.48
Bounded buffer with transactions

In some speculation-based systems, a transaction that needs to wait for some precondition can "deliberately" abort itself with an explicit retry primitive. The system will refrain from restarting the transaction until some previously read location has been changed by another thread. Transactional code for a bounded buffer would be very similar to that of Figure 13.18. We would simply replace

```
region buffer when full_slots < SIZE
    ...
```

and

```
region buffer when full_slots > 0
    ...
```

with

```
atomic                                          atomic
    if full_slots = SIZE then retry      and        if full_slots = 0 then retry
    ...                                             ...
```

TM avoids the need to specify object(s) on which to implement mutual exclusion. It also allows the condition test to be placed anywhere inside the atomic block. ■

Many different implementations of TM have been proposed, both in hardware and in software. As of 2015, hardware support is commercially available in IBM's z and p series machines, and in Intel's recent versions of the x86. Language-level support is available in Haskell and in several experimental languages and dialects. A formal proposal for transactional extensions to C++ is under consideration for the next revision of the language, expected in 2017 [Int15]. It will be several years before we know how much TM can simplify concurrency in practice, but current signs are promising.

An Example Implementation

There is a surprising amount of variety among software TM systems. We outline one possible implementation here, based, in large part, on the TL2 system of Dice et al. [DSS06] and the TinySTM system of Riegel et al. [FRF08].

Every active transaction keeps track of the locations it has read and the locations and values it has written. It also maintains a valid_time value that indicates the most recent *logical time* at which all of the values it has read so far were known to be correct. Times are obtained from a global clock variable that increases by one each time a transaction attempts to commit. Finally, threads share a global table of *ownership records* (orecs), indexed by hashing the address of a shared location. Each orec contains either (1) the most recent logical time at which any of the locations covered by (hashing to) that orec was updated, or (2) the ID t of a transaction that is currently trying to commit a change to one of those locations. In case (1), the orec is said to be *unowned*; in case (2) the orec—and, by extension, all locations that hash to it—is said to be *owned* by t.

EXAMPLE 13.49
Translation of an atomic block

The compiler translates each atomic block into code roughly equivalent to the following:

```
loop
    valid_time := clock
    read_set := write_map := ▫
    try
        -- your code here
        commit()
        break
    except when abort
        -- continue loop
```

In the body of the transaction (your code here), reads and writes of a location with address x are replaced with calls to read(x) and write(x, v), using the code

```
struct orec
    owned : Boolean
    val : union (time, transaction_id)

function read(x : address) : value
    if x ∈ write_map.domain then return write_map[x]
    loop
        repeat
            o : orec := orecs[hash(x)]
        until not o.owned
        t : time := o.val    -- when last modified
        if t > valid_time
            -- may be inconsistent with previous reads
            validate()       -- attempt to extend valid_time
        v : value := *x
        if o = orecs[hash(x)]
            read_set +:= {x}
            return v

procedure validate()
    t : time := clock
    for x : address ∈ read_set
        o : orec := orecs[hash(x)]
        if (not o.owned and o.val > valid_time)
                or (o.owned and o.val ≠ me)
            throw abort
    valid_time := t

procedure write(x : address, v : value)
    write_map[x] := v

procedure commit()
    try
        lock_map : map address → orec := ▫
        done : Boolean := false
        for x : address ∈ write_map.domain
            o : orec := orecs[hash(x)]
            if o ≠ ⟨true, me⟩
                if o.owned then throw abort
                if not CAS(&orecs[hash(x)], o, ⟨true, me⟩)
                    throw abort
                lock_map[x] := o
        n : time := 1 + fetch_and_increment(&clock)
        validate()
        done := true
        for ⟨x, v⟩ : ⟨address, value⟩ ∈ write_map
            *x = v                      -- write back
    finally
        -- do this however control leaves the try block
        for ⟨x, o⟩ : ⟨address, orec⟩ ∈ lock_map
            orecs[hash(x)] := if done
                then ⟨false, n⟩         -- update
                else o                  -- restore
```

Figure 13.19 Possible pseudocode for a software TM system. The read and write routines are used to replace ordinary loads and stores within the body of the transaction. The validate routine is called from both read and commit. It attempts to verify that no previously read value has since been overwritten and, if successful, updates valid_time. Various fence instructions (not shown) may be needed if the underlying hardware is not sequentially consistent.

shown in Figure 13.19. Also shown is the commit routine, called at the end of the try block above. ■

Briefly, a transaction buffers its (speculative) writes until it is ready to commit. It then locks all the locations it needs to write, verifies that all the locations it previously read have not been overwritten since, and then writes back and unlocks the locations. At all times, the transaction knows that all of its reads were mutually consistent at time valid_time. If it ever tries to read a new location that has been updated since valid_time, it attempts to *extend* this time to the current value of the global clock. If it is able to perform a similar extension at commit time, after having locked all locations it needs to change, then the aggregate effect of the transaction as a whole will be as if it had occurred instantaneously at commit time.

To implement retry (not shown in Figure 13.19), we can add an optional list of threads to every orec. A retrying thread will add itself to the list of every location

in its `read_set` and then perform a P operation on a thread-specific semaphore. Meanwhile, any thread that commits a change to an orec with waiting threads performs a V on the semaphore of each of those threads. This mechanism will sometimes result in unnecessary wakeups, but these do not impact correctness. Upon wakeup, a thread removes itself from all thread lists before restarting its transaction.

Challenges

Many subtleties have been glossed over in our example implementation. The translation in Example 13.49 will not behave correctly if code inside the atomic block throws an exception (other than abort) or executes a return or an exit out of some surrounding loop. The pseudocode of Figure 13.19 also fails to consider that transactions may be nested.

Several additional issues are still the subject of debate among TM designers. What should we do about operations inside transactions (I/O, system calls, etc.) that cannot easily be rolled back, and how do we prevent such transactions from ever calling retry? How do we discourage programmers from creating transactions so large they almost always conflict with one another, and cannot run in parallel? Should a program ever be able to detect that transactions are aborting? How should transactions interact with locks and with nonblocking data structures? Should races between transactions and nontransactional code be considered program bugs? If so, should there be any constraints on the behavior that may result? These and similar questions will need to be answered by any production-quality TM-capable language.

13.4.5 Implicit Synchronization

In several shared-memory languages, the operations that threads can perform on shared data are restricted in such a way that synchronization can be implicit in the operations themselves, rather than appearing as separate, explicit operations. We have seen one example of implicit synchronization already: the `forall` loop of HPF and Fortran 95 (Example 13.10). Separate iterations of a `forall` loop proceed concurrently, semantically in lock-step with each other: each iteration reads all data used in its instance of the first assignment statement before any iteration updates its instance of the left-hand side. The left-hand side updates in turn occur before any iteration reads the data used in its instance of the second assignment statement, and so on. Compilation of `forall` loops for vector machines, while far from trivial, is more or less straightforward. On a more conventional multiprocessor, however, good performance usually depends on high-quality *dependence analysis*, which allows the compiler to identify situations in which statements within a loop do not in fact depend on one another, and can proceed without synchronization.

Dependence analysis plays a crucial role in other languages as well. In Sidebar 11.1 we mentioned the purely functional languages Sisal and pH (recall that

iterative constructs in these languages are syntactic sugar for tail recursion). Because these languages are side-effect free, their constructs can be evaluated in any order, or concurrently, as long as no construct attempts to use a value that has yet to be computed. The Sisal implementation developed at Lawrence Livermore National Lab used extensive compiler analysis to identify promising constructs for parallel execution. It also employed tags on data objects that indicate whether the object's value had been computed yet. When the compiler was unable to guarantee that a value would have been computed by the time it was needed at run time, the generated code used tag bits for synchronization, spinning or blocking until they were properly set. Sisal's developers claimed (in 1992) that their language and compiler rivaled parallel Fortran in performance [Can92].

Automatic parallelization, first for vector machines and then for general-purpose machines, was a major topic of research in the 1980s and 1990s. It achieved considerable success with well-structured data-parallel programs, largely for scientific applications, and largely but not entirely in Fortran. Automatic identification of thread-level parallelism in more general, irregularly structured programs proved elusive, however, as did compilation for message-passing hardware. Research in this area continues, and has branched out to languages like Matlab and R.

Futures

EXAMPLE 13.50
`future` construct in Multilisp

Implicit synchronization can also be achieved without compiler analysis. The Multilisp [Hal85, MKH91] dialect of Scheme allowed the programmer to enclose any function evaluation in a special `future` construct:

```
(future (my-function my-args))
```

In a purely functional program, `future` is semantically neutral: assuming all evaluations terminate, program behavior will be exactly the same as if `(my-function my-args)` had appeared without the surrounding call. In the implementation, however, `future` arranges for the embedded function to be evaluated by a separate thread of control. The parent thread continues to execute until it actually tries to use the return value of `my-function`, at which point it waits for execution of the `future` to complete. If two or more arguments to a function are enclosed in `future`s, then evaluation of the arguments can proceed in parallel:

```
(parent (future (child1 args1 )) (future (child2 args2 )))
```

There were no additional synchronization mechanisms in Multilisp: `future` itself was the language's only addition to Scheme. Many subsequent languages and systems have provided `future` as part of a larger feature set. Using C#'s Task Parallel Library (TPL), we might write

EXAMPLE 13.51
Futures in C#

```
var description = Task.Factory.StartNew(() => GetDescription());
var numberInStock = Task.Factory.StartNew(() => GetInventory());
...
Console.WriteLine("We have " + numberInStock.Result
    + " copies of " + description.Result + " in stock");
```

Static library class `Task.Factory` is used to generate futures, known as "tasks" in C#. The `Create` method supports generic type inference, allowing us to pass a delegate compatible with `Func<T>` (function returning `T`), for any `T`. We've specified the delegates here as lambda expressions. If `GetDescription` returns a `String`, `description` will be of type `Task<String>`; if `GetInventory` returns an `int`, `numberInStock` will be of type `Task<int>`.

The Java standard library provides similar facilities, but the lack of delegates, properties (like `Result`), type inference (`var`), and automatic boxing (of the `int` returned by `GetInventory`) make the syntax quite a bit more cumbersome. Java also requires that the programmer pass newly created `Futures` to an explicitly created `Executor` object that will be responsible for running them. Scala provides syntax for futures as simple as that of C#, with even richer semantics. ■

EXAMPLE 13.52
Futures in C++11

Futures are also available in C++, where they are designed to interoperate with lambda expressions, object closures, and a variety of mechanisms for threading and asynchronous (delayed) computation. Perhaps the simplest use case employs the generic `async` function, which takes a function `f` and a list of arguments a_1, ..., a_n, and returns a future that will eventually yield `f(`a_1`, ..., `a_n`)`:

```
string ip_address_of(const char* hostname) {
    // do Internet name lookup (potentially slow)
}
...
```

DESIGN & IMPLEMENTATION

13.9 Side-effect freedom and implicit synchronization

In a partially imperative program (in Multilisp, C#, Scala, etc.), the programmer must take care to make sure that concurrent execution of `futures` will not compromise program correctness. The expression (`parent (future (child1` *args1*`)) (future (child2` *args2*`)))` may produce unpredictable behavior if the evaluations of `child1` and `child2` depend on one another, or if the evaluation of `parent` depends on any aspect of `child1` and `child2` other than their return values. Such behavior may be very difficult to debug. Languages like Sisal and Haskell avoid the problem by permitting only side-effect–free programs.

In a key sense, pure functional languages are ideally suited to parallel execution: they eliminate all artificial connections—all anti- and output dependences (Section C-17.6)—among expressions: all that remains is the actual *data flow*. Two principal barriers to performance remain: (1) the standard challenges of efficient code generation for functional programs (Section 11.8), and (2) the need to identify which potentially parallel code fragments are large enough and independent enough to merit the overhead of thread creation and implicit synchronization.

```
auto query = async(ip_address_of, "www.cs.rochester.edu");
...
cout << query.get() << "\n";    // prints "192.5.53.208"
```

Here variable `query`, which we declared with the `auto` keyword, will be inferred to have type `future<string>`.

In some ways the `future` construct of Multilisp resembles the built-in `delay` and `force` of Scheme (Section 6.6.2). Where `future` supports concurrency, however, `delay` supports lazy evaluation: it defers evaluation of its embedded function until the return value is known to be needed. Any use of a `delay`ed expression in Scheme must be surrounded by `force`. By contrast, synchronization on a Multilisp `future` is implicit—there is no analog of `force`.

A more complicated variant of the C++ `async`, not used in Example 13.52, allows the programmer to insist that the future be run in a separate thread—or, alternatively, that it remain unevaluated until `get` is called (at which point it will execute in the calling thread). When `async` is used as shown in our example, the choice of implementation is left to the run-time system—as it is in Multilisp.

Parallel Logic Programming

Several researchers have noted that the backtracking search of logic languages such as Prolog is also amenable to parallelization. Two strategies are possible. The first is to pursue in parallel the subgoals found in the right-hand side of a rule. This strategy is known as *AND parallelism*. The fact that variables in logic, once initialized, are never subsequently modified ensures that parallel branches of an AND cannot interfere with one another. The second strategy is known as *OR parallelism*; it pursues alternative resolutions in parallel. Because they will generally employ different unifications, branches of an OR must use separate copies of their variables. In a search tree such as that of Figure 12.1, AND parallelism and OR parallelism create new threads at alternating levels.

OR parallelism is *speculative*: since success is required on only one branch, work performed on other branches is in some sense wasted. OR parallelism works well, however, when a goal cannot be satisfied (in which case the entire tree must be searched), or when there is high variance in the amount of execution time required to satisfy a goal in different ways (in which case exploring several branches at once reduces the expected time to find the first solution). Both AND and OR parallelism are problematic in Prolog, because they fail to adhere to the deterministic search order required by language semantics. Parlog [Che92], which supports both AND and OR parallelism, is the best known of the parallel Prolog dialects.

CHECK YOUR UNDERSTANDING

38. What is a *monitor*? How do monitor *condition variables* differ from semaphores?
39. Explain the difference between treating monitor signals as *hints* and treating them as *absolutes*.
40. What is a *monitor invariant*? Under what circumstances must it be guaranteed to hold?
41. Describe the *nested monitor problem* and some potential solutions.
42. What is *deadlock*?
43. What is a *conditional critical region*? How does it differ from a monitor?
44. Summarize the synchronization mechanisms of Ada 95, Java, and C#. Contrast them with one another, and with monitors and conditional critical regions. Be sure to explain the features added to Java 5.
45. What is *transactional memory*? What advantages does it offer over algorithms based on locks? What challenges will need to be overcome before it enters widespread use?
46. Describe the semantics of the HPF/Fortran 95 `forall` loop. How does it differ from `do concurrent`?
47. Why might pure functional languages be said to provide a particularly attractive setting for concurrent programming?
48. What are *futures*? In what languages do they appear? What precautions must the programmer take when using them?
49. Explain the difference between AND *parallelism* and OR *parallelism* in Prolog.

13.5 Message Passing

Shared-memory concurrency has become ubiquitous on multicore processors and multiprocessor servers. Message passing, however, still dominates both distributed and high-end computing. Supercomputers and large-scale clusters are programmed primarily in Fortran or C/C++ with the MPI library package. Distributed computing increasingly relies on client–server abstractions layered on top of libraries that implement the TCP/IP Internet standard. As in shared-memory computing, scores of message-passing languages have also been developed for particular application domains, or for research or pedagogical purposes.

IN MORE DEPTH

Three central issues in message-based concurrency—naming, sending, and receiving—are explored on the companion site. A name may refer directly to a process, to some communication resource associated with a process (often called an *entry* or *port*), or to an independent *socket* or *channel* abstraction. A *send* operation may be entirely asynchronous, in which case the sender continues while the underlying system attempts to deliver the message, or the sender may wait, typically for acknowledgment of receipt or for the return of a reply. A *receive* operation, for its part, may be executed explicitly, or it may implicitly trigger execution of some previously specified handler routine. When implicit receipt is coupled with senders waiting for replies, the combination is typically known as *remote procedure call* (RPC). In addition to message-passing libraries, RPC systems typically rely on a language-aware tool known as a *stub compiler*.

13.6 Summary and Concluding Remarks

Concurrency and parallelism have become ubiquitous in modern computer systems. It is probably safe to say that most computer research and development today involves concurrency in one form or another. High-end computer systems have always been parallel, and multicore PCs and cellphones are now ubiquitous. Even on uniprocessors, graphical and networked applications are typically concurrent.

In this chapter we have provided an introduction to concurrent programming with an emphasis on programming language issues. We began with an overview of the motivations for concurrency and of the architecture of modern multiprocessors. We then surveyed the fundamentals of concurrent software, including communication, synchronization, and the creation and management of threads. We distinguished between shared-memory and message-passing models of communication and synchronization, and between language- and library-based implementations of concurrency.

Our survey of thread creation and management described some six different constructs for creating threads: co-begin, parallel loops, launch-at-elaboration, fork/join, implicit receipt, and early reply. Of these fork/join is the most common; it is found in a host of languages, and in library-based packages such as MPI and OpenMP. RPC systems typically use fork/join internally to implement implicit receipt. Regardless of the thread-creation mechanism, most concurrent programming systems implement their language- or library-level threads on top of a collection of OS-level processes, which the operating system implements in a similar manner on top of a collection of hardware cores. We built our sample implementation in stages, beginning with coroutines on a uniprocessor, then adding a ready list and scheduler, then timers for preemption, and finally parallel scheduling on multiple cores.

The bulk of the chapter focused on shared-memory programming models, and on synchronization in particular. We distinguished between atomicity and condition synchronization, and between busy-wait and scheduler-based implementations. Among busy-wait mechanisms we looked in particular at spin locks and barriers. Among scheduler-based mechanisms we looked at semaphores, monitors, and conditional critical regions. Of the three, semaphores are the simplest, and remain widely used, particularly in operating systems. Monitors and conditional critical regions provide better encapsulation and abstraction, but are not amenable to implementation in a library. Conditional critical regions might be argued to provide the most pleasant programming model, but cannot in general be implemented as efficiently as monitors.

We also considered the implicit synchronization provided by parallel functional languages and by parallelizing compilers for such data-parallel languages as High Performance Fortran. For programs written in a functional style, we considered the `future` mechanism introduced by Multilisp and subsequently incorporated into many other languages, including Java, C#, C++, and Scala.

As an alternative to lock-based atomicity, we considered nonblocking data structures, which avoid performance anomalies due to inopportune preemption and page faults. For certain common structures, nonblocking algorithms can outperform locks even in the common case. Unfortunately, they tend to be extraordinarily subtle and difficult to create.

Transactional memory (TM) was originally conceived as a general-purpose means of building nonblocking code for arbitrary data structures. Most recent implementations, however, have given up on nonblocking guarantees, focusing instead on the ability to specify atomicity without devising an explicit locking protocol. Like conditional critical regions, TM sacrifices performance for the sake of programmability. Prototype implementations are now available for a wide variety of languages, with hardware support in several commercial instruction sets.

Our section on message passing, mostly on the companion site, drew examples from several libraries and languages, and considered how processes name each other, how long they block when sending a message, and whether receipt is implicit or explicit. Distributed computing increasingly relies on remote procedure calls, which combine remote-invocation `send` (wait for a reply) with implicit message receipt.

As in previous chapters, we saw many cases in which language design and language implementation influence one another. Some mechanisms (cactus stacks, conditional critical regions, content-based message screening) are sufficiently complex that many language designers have chosen not to provide them. Other mechanisms (Ada-style parameter modes) have been developed specifically to facilitate an efficient implementation technique. And in still other cases (the semantics of no-wait send, blocking inside a monitor), implementation issues play a major role in some larger set of tradeoffs.

Despite the very long history of concurrent language design, until recently most multithreaded programs relied on library-based thread packages. Even C and C++ are now explicitly parallel, however, and it is hard to imagine any new

languages being designed for purely sequential execution. As of 2015, explicitly parallel languages have yet to seriously undermine the dominance of MPI for high-end scientific computing, though this, too, may change in coming years.

13.7 Exercises

13.1 Give an example of a "benign" race condition—one whose outcome affects program behavior, but not correctness.

13.2 We have defined the *ready list* of a thread package to contain all threads that are runnable but not running, with a separate variable to identify the currently running thread. Could we just as easily have defined the ready list to contain *all* runnable threads, with the understanding that the one at the head of the list is running? (Hint: Think about multiprocessors.)

13.3 Imagine you are writing the code to manage a hash table that will be shared among several concurrent threads. Assume that operations on the table need to be atomic. You could use a single mutual exclusion lock to protect the entire table, or you could devise a scheme with one lock per hash-table bucket. Which approach is likely to work better, under what circumstances? Why?

13.4 The typical spin lock holds only one bit of data, but requires a full word of storage, because only full words can be read, modified, and written atomically in hardware. Consider, however, the hash table of the previous exercise. If we choose to employ a separate lock for each bucket of the table, explain how to implement a "two-level" locking scheme that couples a conventional spin lock for the table as a whole with a *single bit* of locking information for each bucket. Explain why such a scheme might be desirable, particularly in a table with external chaining.

13.5 Drawing inspiration from Examples 13.29 and 13.30, design a non-blocking linked-list implementation of a stack using `compare_and_swap`. (When `CAS` was first introduced, on the IBM 370 architecture, this algorithm was one of the driving applications [Tre86].)

13.6 Building on the previous exercise, suppose that stack nodes are dynamically allocated. If we read a pointer and then are delayed (e.g., due to preemption), the node to which the pointer refers may be reclaimed and then reallocated for a different purpose. A subsequent `compare-and-swap` may then succeed when logically it should not. This issue is known as the *ABA problem*.

Give a concrete example—an interleaving of operations in two or more threads—where the ABA problem may result in incorrect behavior for your stack. Explain why this behavior cannot occur in systems with automatic garbage collection. Suggest what might be done to avoid it in systems with manual storage management.

13.7 We noted in Section 13.3.2 that several processors, including the ARM, MIPS, and Power, provide an alternative to `compare_and_swap` (`CAS`) known as `load_linked/store_conditional` (`LL/SC`). A `load_linked` instruction loads a memory location into a register and stores certain bookkeeping information into hidden processor registers. A `store_conditional` instruction stores the register back into the memory location, but only if the location has not been modified by any other processor since the `load_linked` was executed. Like `compare_and_swap`, `store_conditional` returns an indication of whether it succeeded or not.

(a) Rewrite the code sequence of Example 13.29 using `LL/SC`.

(b) On most machines, an `SC` instruction can fail for any of several "spurious" reasons, including a page fault, a cache miss, or the occurrence of an interrupt in the time since the matching `LL`. What steps must a programmer take to make sure that algorithms work correctly in the face of such failures?

(c) Discuss the relative advantages of `LL/SC` and `CAS`. Consider how they might be implemented on a cache-coherent multiprocessor. Are there situations in which one would work but the other would not? (Hints: Consider algorithms in which a thread may need to touch more than one memory location. Also consider algorithms in which the contents of a memory location might be changed and then restored, as in the previous exercise.)

13.8 Starting with the test-and-`test_and_set` lock of Figure 13.8, implement busy-wait code that will allow readers to access a data structure concurrently. Writers will still need to lock out both readers and other writers. You may use any reasonable atomic instruction(s) (e.g., `LL/SC`). Consider the issue of fairness. In particular, if there are *always* readers interested in accessing the data structure, your algorithm should ensure that writers are not locked out forever.

13.9 Assuming the Java memory model,

(a) Explain why it is not sufficient in Figure 13.11 to label `X` and `Y` as `volatile`.

(b) Explain why it *is* sufficient, in that same figure, to enclose `C`'s reads (and similarly those of `D`) in a `synchronized` block for some common shared object `O`.

(c) Explain why it is sufficient, in Example 13.31, to label both `inspected` and `X` as `volatile`, but not to label only one.

(Hint: You may find it useful to consult Doug Lea's Java Memory Model "Cookbook for Compiler Writers," at *gee.cs.oswego.edu/dl/jmm/cookbook.html*).

13.10 Implement the nonblocking queue of Example 13.30 on an x86. (Complete pseudocode can be found in the paper by Michael and Scott [MS98].)

Do you need fence instructions to ensure consistency? If you have access to appropriate hardware, port your code to a machine with a more relaxed memory model (e.g., ARM or Power). What new fences or atomic references do you need?

13.11 Consider the implementation of software transactional memory in Figure 13.19.

(a) How would you implement the read_set, write_map, and lock_map data structures? You will want to minimize the cost not only of insert and lookup operations but also of (1) "zeroing out" the table at the end of a transaction, so it can be used again; and (2) extending the table if it becomes too full.

(b) The validate routine is called in two different places. Expand these calls in-line and customize them to the calling context. What optimizations can you achieve?

(c) Optimize the commit routine to exploit the fact that a final validation is unnecessary if no other transaction has committed since valid_time.

(d) Further optimize commit by observing that the for loop in the finally clause really needs to iterate over orecs, not over addresses (there may be a difference, if more than one address hashes to the same orec). What data, ideally, should lock_map hold?

13.12 The code of Example 13.35 could fairly be accused of displaying poor abstraction. If we make *desired_condition* a delegate (a subroutine or object closure), can we pass it as an extra parameter, and move the signal and scheduler_lock management inside sleep_on? (Hint: Consider the code for the P operation in Figure 13.15.)

13.13 The mechanism used in Figure 13.13 to make scheduler code reentrant employs a single OS-provided lock for all the scheduling data structures of the application. Among other things, this mechanism prevents threads on separate processors from performing P or V operations on unrelated semaphores, even when none of the operations needs to block. Can you devise another synchronization mechanism for scheduler-related operations that admits a higher degree of concurrency but that is still correct?

13.14 Show how to implement a lock-based concurrent set as a singly linked sorted list. Your implementation should support insert, find, and remove operations, and should permit operations on separate portions of the list to occur concurrently (so a single lock for the entire list will not suffice). (Hint: You will want to use a "walking lock" idiom in which acquire and release operations are interleaved in non-LIFO order.)

13.15 (Difficult) Implement a nonblocking version of the set of the previous exercise. (Hint: You will probably discover that insertion is easy but deletion is hard. Consider a *lazy deletion* mechanism in which cleanup [physical removal of a node] may occur well after logical completion of the removal. For further details see the work of Harris [Har01].)

13.16 To make spin locks useful on a multiprogrammed multiprocessor, one might want to ensure that no process is ever preempted in the middle of a critical section. That way it would always be safe to spin in user space, because the process holding the lock would be guaranteed to be running on some other processor, rather than preempted and possibly in need of the current processor. Explain why an operating system designer might not want to give user processes the ability to disable preemption arbitrarily. (Hint: Think about fairness and multiple users.) Can you suggest a way to get around the problem? (References to several possible solutions can be found in the paper by Kontothanassis, Wisniewski, and Scott [KWS97].)

13.17 Show how to use semaphores to construct a scheduler-based n-thread barrier.

13.18 Prove that monitors and semaphores are equally powerful. That is, use each to implement the other. In the monitor-based implementation of semaphores, what is your monitor invariant?

13.19 Show how to use binary semaphores to implement general semaphores.

13.20 In Example 13.38 (Figure 13.15), suppose we replaced the middle four lines of procedure P with

```
if S.N = 0
    sleep_on(S.Q)
S.N −:= 1
```

and the middle four lines of procedure V with

```
S.N +:= 1
if S.Q is nonempty
    enqueue(ready_list, dequeue(S.Q))
```

What is the problem with this new version? Explain how it connects to the question of hints and absolutes in Section 13.4.1.

13.21 Suppose that every monitor has a separate mutual exclusion lock, so that different threads can run in different monitors concurrently, and that we want to release exclusion on both inner and outer monitors when a thread `waits` in a nested call. When the thread awakens it will need to reacquire the outer locks. How can we ensure its ability to do so? (Hint: Think about the order in which to acquire locks, and be prepared to abandon Hoare semantics. For further hints, see Wettstein [Wet78].)

13.22 Show how general semaphores can be implemented with conditional critical regions in which all threads wait for the same condition, thereby avoiding the overhead of unproductive wake-ups.

13.23 Write code for a bounded buffer using the protected object mechanism of Ada 95.

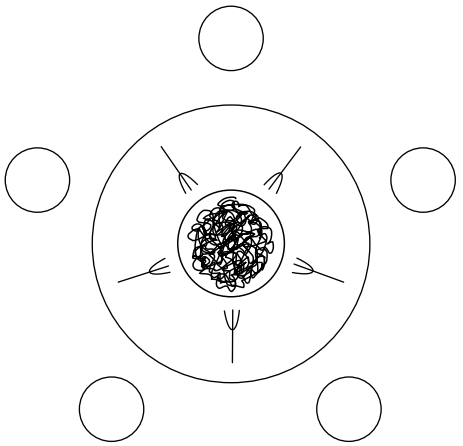


Figure 13.20 The Dining Philosophers. Hungry philosophers must contend for the forks to their left and right in order to eat.

13.24 Repeat the previous exercise in Java using `synchronized` statements or methods. Try to make your solution as simple and conceptually clear as possible. You will probably want to use `notifyAll`.

13.25 Give a more efficient solution to the previous exercise that avoids the use of `notifyAll`. (*Warning:* It is tempting to observe that the buffer can never be both full and empty at the same time, and to assume therefore that waiting threads are either all producers or all consumers. This need not be the case, however: if the buffer ever becomes even a temporary performance bottleneck, there may be an arbitrary number of waiting threads, including both producers and consumers.)

13.26 Repeat the previous exercise using Java `Lock` variables.

13.27 Explain how *escape analysis*, mentioned briefly in Sidebar 10.3, could be used to reduce the cost of certain `synchronized` statements and methods in Java.

13.28 The *dining philosophers problem* [Dij72] is a classic exercise in synchronization (Figure 13.20). Five philosophers sit around a circular table. In the center is a large communal plate of spaghetti. Each philosopher repeatedly thinks for a while and then eats for a while, at intervals of his or her own choosing. On the table between each pair of adjacent philosophers is a single fork. To eat, a philosopher requires both adjacent forks: the one on the left and the one on the right. Because they share a fork, adjacent philosophers cannot eat simultaneously.

Write a solution to the dining philosophers problem in which each philosopher is represented by a process and the forks are represented by shared data. Synchronize access to the forks using semaphores, monitors, or conditional critical regions. Try to maximize concurrency.

13.29 In the previous exercise you may have noticed that the dining philosophers are prone to deadlock. One has to worry about the possibility that all five of them will pick up their right-hand forks simultaneously, and then wait forever for their left-hand neighbors to finish eating.

Discuss as many strategies as you can think of to address the deadlock problem. Can you describe a solution in which it is provably impossible for any philosopher to go hungry forever? Can you describe a solution that is fair in a strong sense of the word (i.e., in which no one philosopher gets more chance to eat than some other over the long term)? For a particularly elegant solution, see the paper by Chandy and Misra [CM84].

13.30 In some concurrent programming systems, global variables are shared by all threads. In others, each newly created thread has a separate copy of the global variables, commonly initialized to the values of the globals of the creating thread. Under this private globals approach, shared data must be allocated from a special heap. In still other programming systems, the programmer can specify which global variables are to be private and which are to be shared.

Discuss the tradeoffs between private and shared global variables. Which would you prefer to have available, for which sorts of programs? How would you implement each? Are some options harder to implement than others? To what extent do your answers depend on the nature of processes provided by the operating system?

13.31 Rewrite Example 13.51 in Java.

13.32 AND parallelism in logic languages is analogous to the parallel evaluation of arguments in a functional language (e.g., Multilisp). Does OR parallelism have a similar analog? (Hint: Think about special forms [Section 11.5].) Can you suggest a way to obtain the effect of OR parallelism in Multilisp?

13.33 In Section 13.4.5 we claimed that both AND parallelism and OR parallelism were problematic in Prolog, because they failed to adhere to the deterministic search order required by language semantics. Elaborate on this claim. What specifically can go wrong?

13.34–13.38 In More Depth.

13.8 Explorations

13.39 The MMX, SSE, and AVX extensions to the x86 instruction set and the AltiVec extensions to the Power instruction set make vector operations available to general-purpose code. Learn about these instructions and research their history. What sorts of code are they used for? How are they related to vector supercomputers? To modern graphics processors?

13.40 The "Top 500" list (*top500.org*) maintains information, over time, on the 500 most powerful computers in the world, as measured on the Linpack performance benchmark. Explore the site. Pay particular attention to the historical trends in the kinds of machines deployed. Can you explain these trends? How many cases can you find of supercomputer technology moving into the mainstream, and vice versa?

13.41 In Section 13.3.3 we noted that different processors provide different levels of memory consistency and different mechanisms to force additional ordering when needed. Learn more about these hardware memory models. You might want to start with the tutorial by Adve and Gharachorloo [AG96].

13.42 In Sections 13.3.3 and 13.4.3 we presented a very high-level summary of the Java and C++ memory models. Learn their details. Also investigate the (more loosely specified) models of Ada and C#. How do these compare? How efficiently can each be implemented on various real machines? What are the challenges for implementors? For Java, explore the controversy that arose around the memory model in the original definition of the language (updated in Java 5—see the paper by Manson et al. [MPA05] for a discussion). For C++, pay particular attention to the ability to specify weakened consistency on loads and stores of `atomic` variables.

13.43 In Section 13.3.2 we presented a brief introduction to the design of *nonblocking* concurrent data structures, which work correctly without locks. Learn more about this topic. How hard is it to write correct nonblocking code? How does the performance compare to that of lock-based code? You might want to start with the work of Michael [MS98] and Sundell [Sun04]. For a more theoretical foundation, start with Herlihy's original article on *wait freedom* [Her91] and the more recent concept of *obstruction freedom* [HLM03], or check out the text by Herlihy and Shavit [HS12].

13.44 As possible improvements to reader-writer locks, learn about *sequence locks* [Lam05] and the *RCU* (read-copy update) synchronization idiom [MAK+01]. Both of these are heavily used in the operating systems community. Discuss the challenges involved in applying them to code written by "nonexperts."

13.45 The first software transactional memory systems grew out of work on nonblocking concurrent data structures, and were in fact nonblocking. Most recent systems, however, are lock based. Read the position paper by Ennals [Enn06] and the more recent papers of Marathe and Moir [MM08] and Tabba et al. [TWGM07]. What do you think? Should TM systems be nonblocking?

13.46 The most widely used language-level transactional memory is the *STM monad* of Haskell, supported by the Glasgow Haskell compiler and runtime system. Read up on its syntax and implementation [HMPH05]. Pay

particular attention to the `retry` and `orElse` mechanisms. Discuss their similarities to—and advantages over—conditional critical regions.

13.47 Study the documentation for some of your favorite library packages (the C and C++ standard libraries, perhaps, or the .NET and Java libraries, or the many available packages for mathematical computing). Which routines can safely be called from a multithreaded program? Which cannot? What accounts for the difference? Why not make all routines thread safe?

13.48 Undertake a detailed study of several concurrent languages. Download implementations and use them to write parallel programs of several different sorts. (You might, for example, try Conway's Game of Life, Delaunay Triangulation, and Gaussian Elimination; descriptions of all of these can easily be found on the Web.) Write a paper about your experience. What worked well? What didn't? Languages you might consider include Ada, C#, Cilk, Erlang, Go, Haskell, Java, Modula-3, Occam, Rust, SR, and Swift. References for all of these can be found in Appendix A.

13.49 Learn about the supercomputing languages discussed in the Bibliographic Notes at the end of the chapter: Co-Array Fortran, Titanium, and UPC; and Chapel, Fortress, and X10. How do these compare to one another? To MPI and OpenMP? To languages with less of a focus on "high-end" computing?

13.50 In the spirit of the previous question, learn about the SHMEM library package, originally developed by Robert Numrich of Cray, Inc., and now standardized as OpenSHMEM (*openshmem.org*). SHMEM is widely used for parallel programming on both large-scale multiprocessors and clusters. It has been characterized as a cross between shared memory and message passing. Is this a fair characterization? Under what circumstances might a `shmem` program be expected to outperform solutions in MPI or OpenMP?

13.51 Much of this chapter has been devoted to the management of races in parallel programs. The complexity of the task suggests a tantalizing question: is it possible to design a concurrent programming language that is powerful enough to be widely useful, and in which programs are inherently race-free? For three very different takes on a (mostly) affirmative answer, see the work of Edward Lee [Lee06], the various concurrent dialects of Haskell [NA01, JGF96], and Deterministic Parallel Java (DPJ) [BAD+09].

13.52–13.54 In More Depth.

13.9 Bibliographic Notes

Much of the early study of concurrency stems from a pair of articles by Dijkstra [Dij68a, Dij72]. Andrews and Schneider [AS83] provided an excellent snapshot of the field in the early 1980s. Holt et al. [HGLS78] is a useful reference for many of the classic problems in concurrency and synchronization.

Peterson's two-process synchronization algorithm appears in a remarkably elegant and readable two-page paper [Pet81]. Lamport's 1978 article on "Time, Clocks, and the Ordering of Events in a Distributed System" [Lam78] argued convincingly that the notion of global time cannot be well defined, and that distributed algorithms must therefore be based on causal *happens before* relationships among individual processes. Reader–writer locks are due to Courtois, Heymans, and Parnas [CHP71]. Java 7 phasers were inspired in part by the work of Shirako et al. [SPSS08]. Mellor-Crummey and Scott [MCS91] survey the principal busy-wait synchronization algorithms and introduce locks and barriers that scale without contention to very large machines.

The seminal paper on lock-free synchronization is that of Herlihy [Her91]. The nonblocking concurrent queue of Example 13.30 is due to Michael and Scott [MS96]. Herlihy and Shavit [HS12] and Scott [Sco13] provide modern, book-length coverage of synchronization and concurrent data structures. Adve and Gharachorloo introduce the notion of hardware memory models [AG96]. Pugh explains the problems with the original Java Memory Model [Pug00]; the revised model is described by Manson, Pugh, and Adve [MPA05]. The memory model for C++11 is described by Boehm and Adve [BA08]. Boehm has argued convincingly that threads cannot be implemented correctly without compiler support [Boe05]. The original paper on transactional memory is by Herlihy and Moss [HM93]. Harris, Larus, and Rajwar provide a book-length survey of the field as of late 2010 [HLR10]. Larus and Kozyrakis provide a briefer overview [LK08].

Two recent generations of parallel languages for high-end computing have been highly influential. The Partitioned Global Address Space (PGAS) languages include Co-Array Fortran (CAF), Unified Parallel C (UPC), and Titanium (a dialect of Java). They support a single global name space for variables, but employ an "extra dimension" of addressing to access data not on the local core. Much of the functionality of CAF has been adopted into Fortran 2008. The so-called HPCS languages—Chapel, Fortress, and X10—build on experience with the PGAS languages, but target a broader range of hardware, applications, and styles of parallelism. All three include transactional features. For all of these, a web search is probably the best source of current information.

MPI [Mes12] is documented in a variety of articles and books. The latest version draws several features from an earlier, competing system known as PVM (Parallel Virtual Machine) [Sun90, GBD+94]. Remote procedure call received increasing attention in the wake of Nelson's doctoral research [BN84]. The Open Network Computing RPC standard is documented in Internet RFC number 1831 [Sri95]. RPC also forms the basis of such higher-level standards as CORBA, COM, JavaBeans, and SOAP.

Software distributed shared memory (S-DSM) was originally proposed by Li as part of his doctoral research [LH89]. The TreadMarks system from Rice University was widely considered the most mature and robust of the various implementations [ACD+96].